

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Гантулгын Батням

**Backend системд зориулсан annotation-д
суурилсан Java framework-ийн загварчлал,
хэрэгжүүлэлт**
**(Design and Implementation of an Annotation-Based Java
Framework for Backend Systems)**

Программ хангамж(D 061302)
Бакалаврын судалгааны ажил

Улаанбаатар

2026 оны 3 дугаар сар

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

**Backend системд зориулсан annotation-д суурилсан Java
framework-ийн загварчлал, хэрэгжүүлэлт**
**(Design and Implementation of an Annotation-Based Java
Framework for Backend Systems)**

Программ хангамж(D 061302)
Бакалаврын судалгааны ажил

Удирдагч: _____ Др. Т.Цэрэннадмид

Гүйцэтгэсэн: _____ Г.Батням (22B1NUM5578)

Улаанбаатар

2026 оны 3 дугаар сар

Зохиогчийн баталгаа

Миний бие Гантулгын Батням ” Backend системд зориулсан annotation-д суурилсан Java framework-ийн загварчлал, хэрэгжүүлэлт” сэдэвтэй судалгааны ажлыг гүйцэтгэсэн болохыг зарлаж дараах зүйлсийг баталж байна:

- Ажил нь бүхэлдээ Монгол Улсын Их Сургуульд дээд боловсролын зэрэг горилохоор дэвшүүлсэн болно.
- Энэ ажлын аль нэг хэсгийг эсвэл бүхлээр нь ямар нэг их, дээд сургуулийн зэрэг горилохоор оруулж байгаагүй болно.
- Бусдын хийсэн ажлаас хуулбарлаагүй, эшлэл, зүүлтийг зохистой хийсэн болно.
- Ажлыг зохиогч би хийсэн ба миний хийсэн ажил, бусдын дэмжлэгийг дипломын ажилд тодорхой тусгасан болно.
- Ажилд тусалсан бүх эх сурвалжид талархаж байна.

Гарын үсэг: _____

Огноо: _____

ГАРЧИГ

Удиртгал	1
1. СЭДВИЙН СУДАЛГАА	5
1.1 Dependency Injection болон Inversion of Control зарчмын судалгаа ..	6
1.2 Annotation болон Reflection механизмын судалгаа	16
1.3 Ижил төрлийн framework-үүдийн судалгаа	32
1.4 Ашиглагдах программчлалын хэлний судалгаа	35
1.5 Ашиглагдах технологи, хэрэгслүүдийн судалгаа	36
1.6 Системийн ерөнхий шийдлийн судалгаа	38
1.7 Бүлгийн дүгнэлт	47
2. СИСТЕМИЙН ШААРДЛАГА	48
2.1 Framework-ийн шаардлагын тодорхойлолт	48
2.2 Функциональ бус шаардлага	51
2.3 Custom annotation-уудын шаардлага	53
2.4 Статик загвар	63
2.5 Динамик загвар	71
3. СИСТЕМИЙН ЗОХИОМЖ	74
3.1 Архитектурын ерөнхий үндэслэл	74
3.2 Сонгосон архитектур: Давхаргат архитектур	74
3.3 Сонгосон архитектурын давуу тал	75
3.4 Layered Architecture Diagram	76
3.5 Activity Diagram	78
3.6 Sequence Diagram	81
3.7 Бүлгийн дүгнэлт	83
4. ХЭРЭГЖҮҮЛЭЛТ	84
4.1 Хэрэгжүүлэлтийн орчин ба бүтэц	84

4.2	Annotation давхаргын хэрэгжүүлэлт	85
4.3	Exception давхаргын хэрэгжүүлэлт	88
4.4	Container давхаргын хэрэгжүүлэлт	89
4.5	Scanner давхаргын хэрэгжүүлэлт	94
4.6	Injector давхаргын хэрэгжүүлэлт	94
4.7	Туршилт	95
4.8	Бүлгийн дүгнэлт	99
5.	ДҮГНЭЛТ	100
	НОМ ЗҮЙ	100
	ХАВСРАЛТ	102
A.	КОДЫН ХЭРЭГЖҮҮЛЭЛТ	103
A.1	<i>ApplicationContext.java</i>	103
A.2	<i>ClassPathScanner.java</i>	105
A.3	<i>DependencyInjector.java</i>	106
A.4	<i>BeanRegistry.java</i>	107
A.5	<i>ContextIntegrationTest.java</i>	108

ЗУРГИЙН ЖАГСААЛТ

1.1	IoC(Inversion of Control) зарчим	6
1.2	DI(Dependency Injection) зарчим	8
1.3	Spring Framework	32
1.4	Google Guice	33
1.5	Jakarta CDI	33
1.6	Java программчлалын хэл	35
1.7	Maven builder	36
1.8	JUnit test	36
1.9	IntelliJ IDEA	37
1.10	Eclipse	37
1.11	Git GitHub	37
1.12	Хэрэгжүүлэх DI Framework-ийн ерөнхий ажиллагааны зураглал	47
2.1	DI framework-ийн багцын диаграмм	65
2.2	Annotation-д суурилсан DI framework-ийн классын диаграмм	70
2.3	Use case	71
3.1	Төслийн архитектур	77
3.2	Activity diagramm	79
3.3	Sequence diagramm	81
4.1	Тестэлсэн байдал	98

ХҮСНЭГТИЙН ЖАГСААЛТ

1.1	Dependency Injection аргуудын харьцуулалт.....	14
1.2	Java хэлний түгээмэл meta-annotation-ууд тэдгээрийн үүрэг.....	17
1.3	@Target annotation-д ашиглагддаг ElementType болон тайлбарууд.....	20
1.4	Retention policy-ийн орчны ялгаа	21
1.5	Түгээмэл DI framework-үүдийн харьцуулалт	34
2.1	Framework-ийн функциональ шаардлага	50
2.2	Framework-ийн функциональ бус шаардлагууд.....	52
2.3	Framework-д хэрэгжүүлэх custom annotation-уудын шаардлага	54
2.4	Framework хэрэгжүүлэх технологийн шаардлага	61
2.5	Багцуудын хоорондын хамаарал.....	66
2.6	ApplicationContext классын КҮХ карт	67
2.7	BeanDefinition классын КҮХ карт.....	68
2.8	ClassPathScanner классын КҮХ карт	68
2.9	DependencyInjector классын КҮХ карт	69
2.10	ScopeType төрлийн КҮХ карт.....	69
2.11	Container эхлүүлэх use case-ийн тодорхойлолт	72
2.12	Bean авах use case-ийн тодорхойлолт	73
4.1	Package-уудын тестийн coverage-ийн дүн	98

Кодын жагсаалт

1.1	Constructor Injection жишээ	9
1.2	Setter Injection жишээ	11
1.3	Field Injection жишээ	13
1.4	Custom annotation жишээ	18
1.5	Marker annotation жишээ	19
1.6	Single-value annotation жишээ	19
1.7	Full annotation жишээ	19
1.8	@Target ашигласан жишээ	20
1.9	@Retention ашигласан жишээ	22
1.10	@Documented ашигласан жишээ	22
1.11	@Inherited ашигласан жишээ	22
1.12	@Repeatable ашигласан жишээ	23
1.13	@Qualifier ашигласан жишээ	26
1.14	@Primary ашигласан жишээ	26
1.15	BeanFactory ашигласан жишээ	38
1.16	ApplicationContext ашигласан жишээ	39
1.17	Component болон Service annotation ашигласан жишээ	40
1.18	XML суурилсан configuration жишээ	41
1.19	XML суурилсан configuration жишээ	41
1.20	Annotation-д суурилсан bean бүртгэлтийн жишээ	42
1.21	Component scan тохиргооны жишээ	42
1.22	Java-д суурилсан configuration жишээ	43
1.23	Java configuration-оос container үүсгэх жишээ	43
1.24	Spring Boot configuration жишээ	43
1.25	Autowired ашигласан dependency injection жишээ	44
1.26	Singleton scope ашигласан жишээ	45
1.27	Prototype scope ашигласан жишээ	45
2.1	Reflection ашиглан классыг динамикаар ачаалах жишээ	55
2.2	Static ба dynamic object creation-ийн ялгаа	56
4.1	@Component annotation-ийн хэрэгжүүлэлт	85
4.2	@Autowired annotation-ийн хэрэгжүүлэлт	86
4.3	@Qualifier annotation-ийн хэрэгжүүлэлт	87
4.4	@Scope annotation-ийн хэрэгжүүлэлт	87
4.5	BeanCreationException классын хэрэгжүүлэлт	88
4.6	NoSuchBeanException классын хэрэгжүүлэлт	88
4.7	CircularDependencyException классын хэрэгжүүлэлт	88
4.8	ScopeType enum классын хэрэгжүүлэлт	89

4.9	BeanDefinition	классын хэрэгжүүлэлт	89
4.10	BeanRegistry	классын хэрэгжүүлэлт	90
4.11	ApplicationContext	классын хэрэгжүүлэлт	92
4.12	ClassPathScanner	классын хэрэгжүүлэлт	94
4.13	DependencyInjector	классын хэрэгжүүлэлт	95
4.14	Prototype болон Qualifier	тестийн хэрэгжүүлэлт	96
4.15	Тойрог хамааралыг илрүүлэх	тест	97
4.16	Bean олдохгүй тохиолдолыг шалгах	тест	97

Удиртгал

Орчин үеийн программ хангамжийн хөгжүүлэлтэд framework-ийн үүрэг улам бүр нэмэгдэж, ялангуяа backend систем боловсруулах үйл явцад чухал байр суурь эзэлж байна. Backend систем хөгжүүлэхэд давтагддаг ажлуудыг автоматжуулах, хөгжүүлэгчийн бүтээмжийг нэмэгдүүлэх, кодын хэмжээг багасгах, мөн системийн уян хатан болон өргөжих чадварыг сайжруулах шаардлага улам бүр өсөж байна.

Java хэл дээр хөгжүүлсэн **Spring Framework** нь annotation-д суурилсан тохиргоо (*annotation-based configuration*) болон **Dependency Injection (DI)** зарчмыг нэвтрүүлснээр дэлхий даяар хамгийн өргөн хэрэглэгддэг backend framework-үүдийн нэг болсон. Тус framework нь **Inversion of Control (IoC)** болон **Dependency Injection (DI)** зэрэг архитектурын оновчтой шийдлүүдийг ашигласнаар хөгжүүлэгчийн бичих шаардлагатай кодын хэмжээг эрс бууруулж, системийн өргөжих чадвар (*scalability*), засварлах болон удирдах боломжийг мэдэгдэхүйц нэмэгдүүлдэг.

Гэсэн хэдий ч ихэнх хөгжүүлэгчид эдгээр framework-үүдийг хэрэглээний түвшинд ашигладаг боловч тэдгээрийн дотоод ажиллагааны үндэс болох **Java Reflection API**, annotation боловсруулах механизм, объектын удирдлагын мөчлөг зэрэг суурь ойлголтуудыг төдийлөн гүнзгий судалдаггүй. Үүний улмаас framework-ийн дотоод архитектур, ажиллах зарчим, өргөтгөх боломжийг бүрэн ойлгохгүй байх тохиолдол түгээмэл ажиглагддаг.

Иймээс backend системийн суурь архитектур, объектын амьдралын мөчлөгийн удирдлага, annotation болон reflection-д суурилсан илрүүлэлт, удирдлагын аргачлалыг онолын болон практик түвшинд судлах шаардлага бий болж байна. Энэ нь framework-ийн цөм зарчим болох **IoC**, **DI**, **annotation** боловсруулалт, **reflection** ашиглалт зэрэг ойлголтуудыг илүү гүнзгий эзэмшихээс гадна backend хөгжүүлэлтийн суурь архитектурын мэдлэгийг бататгах, цаашдын судалгаа болон практик хэрэглээнд ач холбогдолтой юм.

Зорилго

Энэхүү дипломын ажлын зорилго нь Java хэлний annotation болон reflection механизмд тулгуурласан суурь түвшний **Dependency Injection Framework** зохион байгуулж, хэрэгжүүлэх замаар IoC болон DI зарчмын дотоод ажиллагааг судлах, түүний хэрэглээний боломжийг backend хөгжүүлэлтийн орчинд туршин үнэлэхэд оршино.

Зорилтууд

Судалгааны зорилгын хүрээнд дараах зорилтуудыг дэвшүүлж байна:

1. Framework, IoC, DI, annotation, reflection зэрэг онолын үндсэн ойлголтуудыг судлах;
2. Spring Framework-ийн dependency injection болон annotation боловсруулах механизмыг шинжлэх;
3. Java Reflection API ашиглан класс, талбар, гишүүн функцийн мэдээллийг тодорхойлох аргуудыг судлах;
4. Custom annotation-ууд тодорхойлон dependency-ийг автоматаар илрүүлэх боломжийг бүрдүүлэх;
5. IoC container-ийн үндсэн бүтэц болон bean удирдлагын механизмыг зохион бүтээх;
6. суурь түвшний хялбаршуулсан DI framework-ийг хэрэгжүүлж турших;
7. Хэрэгжүүлсэн framework-ийн ажиллагааг жишээ backend орчинд шалгаж, үр дүнг үнэлэх.

Судалгааны объект

Энэхүү судалгааны объект нь Java хэл дээрх annotation болон reflection-д суурилсан dependency удирдлагын механизм бүхий backend framework-ийн дотоод архитектур болно.

Судлагдахуун

Судлагдахуун нь IoC container, Dependency Injection механизм, annotation боловсруулах аргачлал, reflection ашиглан объект үүсгэх болон dependency холбох процесс, мөн эдгээрийг framework хэлбэрээр хэрэгжүүлэх зарчим юм.

Судалгааны арга зүй

Энэхүү судалгаанд харьцуулсан шинжилгээний аргыг ашиглан Spring Framework-ийн dependency injection болон annotation боловсруулах механизмыг судалж, түүнийг хэрэгжүүлсэн DI framework-ийн бүтэц, ажиллагаатай харьцуулан үнэлнэ. Үүнд объект үүсгэх зарчим, dependency холболт, container-ийн үүрэг, annotation таних болон боловсруулах ажиллагаа, өргөтгөх боломж зэрэг үзүүлэлтүүдийг харгалзан ижил болон ялгаатай талыг тодорхойлно. Энэхүү судалгаанд дараах үндсэн арга зүйг ашиглана:

- **Баримт бичиг, эх сурвалжийн судалгаа** – Framework-ийн архитектур, IoC, DI, reflection, annotation-ийн онолын үндсийг судлах;
- **Харьцуулсан шинжилгээ** – Spring Framework-ийн механизмтай харьцуулан судлах;
- **Загварчлал, зохиомжийн арга** – DI framework-ийн бүтэц, бүрэлдэхүүн хэсгүүдийг тодорхойлох;
- **Туршилт, хэрэгжүүлэлт** – Java хэл дээр framework боловсруулж, жишээ систем дээр турших;
- **Үр дүнгийн үнэлгээ** – хэрэгжүүлсэн framework-ийн ажиллагаа, давуу тал, хязгаарлалтыг үнэлэх.

Судалгааны ажлын шинэлэг тал

Энэхүү ажлын шинэлэг тал нь өргөн хэрэглэгддэг Spring Framework-ийн дотоод ажиллагааны зарчмыг зөвхөн онолын түвшинд тайлбарлах бус, annotation болон reflection-д суурилсан суурь түвшний **DI Framework** болгон бие даан хэрэгжүүлж туршсанаар backend framework-ийн үндсэн механизмыг илүү ойлгомжтой, бодит түвшинд харуулахад оршино.

Судалгааны ажлын практик ач холбогдол

Энэхүү судалгааны үр дүнд боловсруулсан framework нь IoC болон DI зарчмын хэрэгжилтийг ойлгоход сургалтын болон туршилтын зориулалтаар ашиглагдах боломжтой. Мөн backend хөгжүүлэлтийн суурь архитектур, dependency удирдлага, annotation болон reflection механизмын талаарх мэдлэгийг гүнзгийрүүлэхэд практик ач холбогдолтой. Цаашлаад энэхүү ажлын үр дүн нь framework-ийн дотоод ажиллагаа, шийдлийг судлах,судалгааны дагуу хэрэгжүүлэн сургалтын хэрэглэгдэхүүн болгон ашиглах үндэс суурь болно.

Дипломын ажлын бүтэц

Энэхүү дипломын ажил нь үндсэндээ онолын судалгаа, системийн зохиомж, хэрэгжүүлэлт, туршилт, үр дүнгийн үнэлгээ гэсэн хэсгүүдээс бүрдэнэ. Нэгдүгээр бүлэгт судалгааны онолын үндэс, framework болон dependency injection-ийн талаар авч үзнэ. Хоёрдугаар бүлэгт системийн шаардлага шинжилгээг гаргах. Гуравдугаар бүлэгт зохиомж, архитектурын шийдлийг тайлбарлана. Дөрөвдүгээр бүлэгт framework-ийн хэрэгжүүлэлт, туршилт, үр дүнг танилцуулж, эцэст нь дүгнэлт, цаашдын хөгжүүлэлтийн боломжийг тусгана.

1. СЭДВИЙН СУДАЛГАА

Орчин үеийн программ хангамжийн хөгжүүлэлтэд framework ашиглах нь хөгжүүлэлтийн бүтээмж, кодын зохион байгуулалт, өргөтгөх боломж болон засвар үйлчилгээний чанарт чухал нөлөө үзүүлдэг. Framework нь давтагддаг үйлдлүүдийг автоматжуулах, нийтлэг архитектурын шийдлийг бэлэн байдлаар санал болгох, хөгжүүлэгчийн бичих шаардлагатай кодын хэмжээг бууруулах үндсэн зорилготой байдаг.

Ялангуяа backend систем боловсруулах орчинд framework-ийн үүрэг илүү тодорхой харагддаг. Учир нь backend хөгжүүлэлтэд объектын удирдлага, өгөгдлийн боловсруулалт, хүсэлт хариуны зохицуулалт, бүрэлдэхүүн хоорондын хамаарал зэрэг асуудлууд байнга давтагдан гарч ирдэг. Ийм нөхцөлд framework ашигласнаар системийн бүтэц илүү цэгцтэй, дахин ашиглагдах боломжтой, өргөтгөхөд хялбар хэлбэрт ордог.

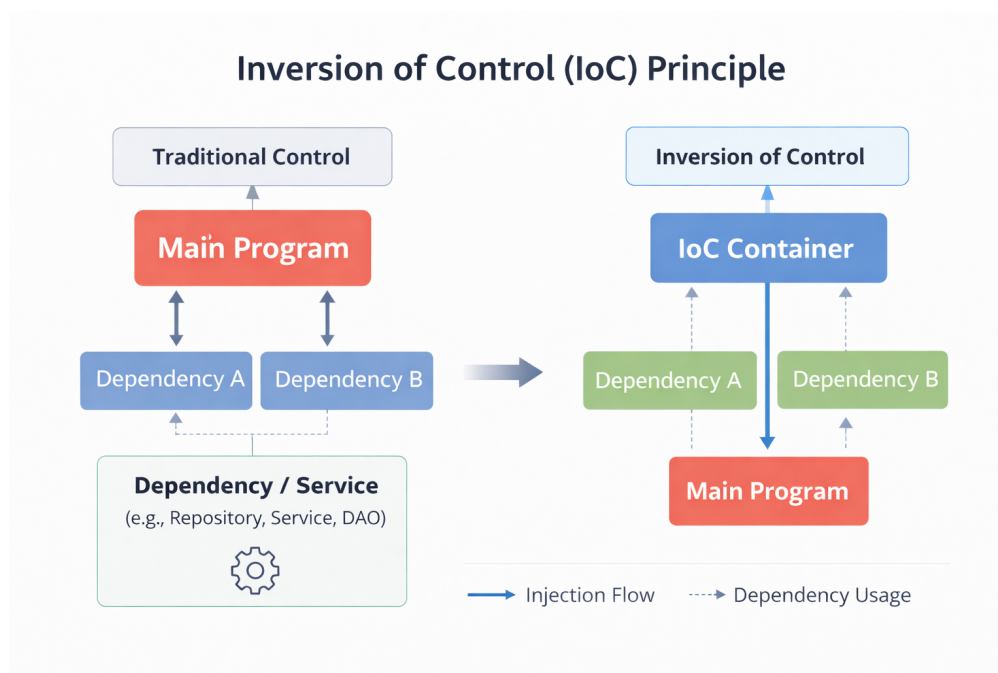
Java экосистемд хамгийн өргөн хэрэглэгддэг backend framework-үүдийн нэг нь Spring Framework юм. Spring нь Inversion of Control, Dependency Injection, annotation-д суурилсан тохиргоо зэрэг зарчмуудыг ашигласнаар хөгжүүлэгчид системийн үндсэн бизнес логик дээр төвлөрөх боломжийг олгодог. Иймээс Spring Framework-ийн дотоод ажиллагааны суурь ойлголтууд болох IoC, DI, annotation, reflection зэрэг механизмуудыг судлах нь энэхүү дипломын ажлын сэдэвтэй шууд холбоотой юм.

1.1 Dependency Injection болон Inversion of Control зарчмын судалгаа

1.1.1 Inversion of Control зарчим

Inversion of Control (IoC) нь объектуудын үүсэлт, удирдлага, хамаарлын шийдлийг программын үндсэн логигоос салгаж, тусгай container эсвэл framework-ээр удирдуулах архитектурын зарчим юм. Уламжлалт аргаар объектууд хоорондоо шууд хамааралтайгаар үүсгэгддэг бол IoC зарчмын үед тухайн удирдлага гаднаас хийгддэг.

IoC зарчмыг ашигласнаар бүрэлдэхүүн хоорондын хамаарал сул болж, кодын уян хатан байдал сайжирдаг. Мөн том хэмжээний системд объектуудын удирдлага төвлөрсөн байдлаар зохион байгуулагддаг тул өргөтгөл, засвар үйлчилгээ хийхэд хялбар болдог.¹



Зураг 1.1: IoC(Inversion of Control) зарчим

Дээрх 1.1-р зурагт уламжлалт удирдлага болон IoC зарчмын үндсэн ялгааг харуулав. Уламжлалт аргаар Main Program нь Dependency A болон Dependency B-г өөрөө үүсгэж, шууд удирддаг.

¹Spring Framework Documentation. *Introduction to the Spring IoC Container and Beans*. <https://docs.spring.io/spring-framework/reference/core/beans/introduction.html>

Харин IoC зарчмын үед IoC Container нь dependency-уудыг удирдаж, Main Program-д шаардлагатай хамаарлыг гаднаас оруулдаг.

1.1.2 Dependency Injection зарчим

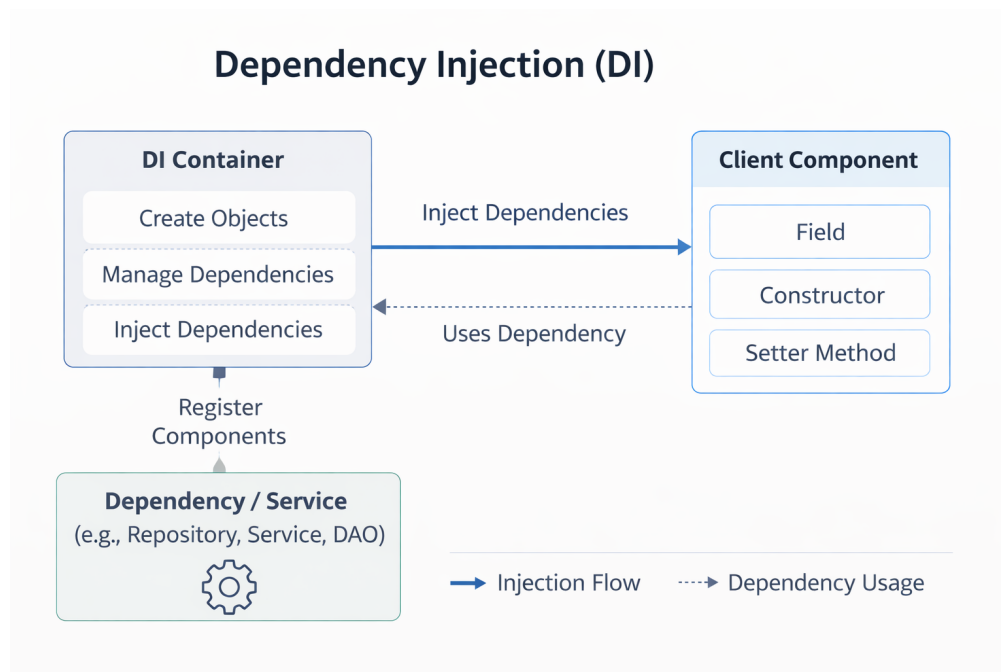
Dependency Injection (DI) нь нэг объектод шаардлагатай өөр объект буюу хамаарлыг тухайн объект өөрөө үүсгэхийн оронд гаднаас оруулах аргачлал юм. Энэ нь **(IoC)** хяналтын удирдлагыг зарчим дээр суурилсан **(design pattern)** үлгэр загвар юм. ²

DI ашигласнаар:

- классуудын хоорондын хамаарал сулрах,
- тестлэх ажиллагаа хялбарших,
- кодын дахин ашиглалт нэмэгдэх,
- системийн өргөтгөх боломж сайжрах

давуу талууд бий болдог.

²Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.



Зураг 1.2: DI(Dependency Injection) зарчим

1.2-р зурагт Dependency Injection-ийн үндсэн механизмыг харуулав. DI Container нь объект үүсгэх, dependency удирдах, inject хийх гэсэн гурван үндсэн үүрэг гүйцэтгэнэ. Dependency / Service буюу Repository, Service, DAO зэрэг бүрэлдэхүүнүүдийг container-д бүртгэсний дараа Client Component-д шаардлагатай dependency-г field, constructor эсвэл setter method хэлбэрээр inject хийдэг.

1.1.3 *Dependency Injection хэрэгжүүлэх үндсэн аргууд*

Dependency Injection нь объект хоорондын хамаарлыг тухайн объект өөрөө үүсгэхийн оронд гаднаас нь нийлүүлэх аргачлал юм. Java болон Spring орчинд dependency injection-ийг хэрэгжүүлэх хамгийн түгээмэл 3 арга байдаг. Үүнд constructor injection, setter injection, field injection орно.

Constructor Injection

Constructor Injection нь объект үүсэх мөчид шаардлагатай бүх хамаарлыг байгуулагч функцээр дамжуулан олгодог арга юм. Энэ арга нь dependency-үүдийг заавал дамжуулах шаардлагатай нөхцөлд хамгийн тохиромжтой бөгөөд immutable бүтэц үүсгэх боломжтой.

```
1 interface UserRepository {
2     String findUserById(int id);
3 }
4
5 class UserRepositoryImpl implements UserRepository {
6     @Override
7     public String findUserById(int id) {
8         return "User:□" + id;
9     }
10 }
11
12 class UserService {
13     private final UserRepository userRepository;
14
15     public UserService(UserRepository userRepository) {
16         this.userRepository = userRepository;
17     }
18
19     public void printUser(int id) {
```

```

20         System.out.println(userRepository.findById(id));
21     }
22 }
23
24 public class Main {
25     public static void main(String[] args) {
26         UserRepository repo = new UserRepositoryImpl();
27         UserService service = new UserService(repo);
28         service.printUser(1);
29     }
30 }

```

Код 1.1: Constructor Injection жишээ

Энэ аргын үед `UserService` класс нь `UserRepository` хамаарлаа байгуулагч функцээрээ дамжуулан авч байна. Иймээс тухайн объект `dependency`-гүйгээр үүсэх боломжгүй болж, илүү найдвартай бүтэц бүрддэг.

Setter Injection

Setter Injection нь объектыг эхлээд үүсгээд, дараа нь setter гишүүн функцийг ашиглан dependency-г оноодог арга юм. Энэ нь optional dependency буюу хамаарал заавал хэрэгтэй бус үед хэрэглэхэд тохиромжтой.

```
1 interface PaymentService {
2     void pay(double amount);
3 }
4
5 class CreditCardPaymentService implements PaymentService {
6     @Override
7     public void pay(double amount) {
8         System.out.println("Paid with credit card:" + amount);
9     }
10 }
11
12 class OrderService {
13     private PaymentService paymentService;
14
15     public void setPaymentService(PaymentService paymentService) {
16         this.paymentService = paymentService;
17     }
18
19     public void checkout(double amount) {
20         paymentService.pay(amount);
21     }
22 }
23
24 public class Main {
25     public static void main(String[] args) {
26         PaymentService payment = new CreditCardPaymentService();
27     }
```

```
28     OrderService orderService = new OrderService();
29     orderService.setPaymentService(payment);
30
31     orderService.checkout(150.0);
32 }
33 }
```

Код 1.2: Setter Injection жишээ

Энэ жишээнд `OrderService` объект эхлээд үүсэж, дараа нь `setPaymentService()` method-оор dependency-г авч байна. Иймээс dependency-г runtime үед өөрчлөх, солих боломжтой байдаг.

Field Injection

Field Injection нь dependency-г класс доторх талбарт шууд оноох арга юм. Framework-үүд, ялангуяа Spring нь annotation ашиглан энэ аргыг хэрэгжүүлдэг. Кодын хувьд богино, ойлгомжтой боловч тестлэх болон dependency-г ил тод харахад сул талтай.

```
1  import org.springframework.beans.factory.annotation.Autowired;
2  import org.springframework.stereotype.Component;
3  import org.springframework.stereotype.Service;
4
5  interface NotificationService {
6      void send(String message);
7  }
8
9  @Component
10 class EmailNotificationService implements NotificationService {
11     @Override
12     public void send(String message) {
13         System.out.println("Email☐sent:☐" + message);
14     }
15 }
16
17 @Service
18 class AlertService {
19
20     @Autowired
21     private NotificationService notificationService;
22
23     public void alertUser() {
24         notificationService.send("System☐alert!");
25     }
```

Код 1.3: Field Injection жишээ

Энэ жишээнд `AlertService` класс доторх `notificationService` талбарт `@Autowired` annotation ашиглан `dependency injection` хийж байна. Хөгжүүлэгч тухайн `dependency`-г гараар `new` ашиглан үүсгэхгүй, харин `framework` автоматаар оноож өгдөг.

1.1.4 Аргуудын харьцуулалт

`Dependency Injection`-ийн дээрх 3 аргыг дараах байдлаар харьцуулан авч үзэж болно.

Table 1.1: `Dependency Injection` аргуудын харьцуулалт

Арга	Давуу тал	Сул тал
Constructor Injection	Dependency заавал олгогдоно, test хийхэд хялбар, immutable бүтэц үүсгэнэ	Dependency олон бол constructor урт болно
Setter Injection	Optional dependency-д тохиромжтой, runtime үед солих боломжтой	Dependency дутуу үлдэх эрсдэлтэй
Field Injection	Код богино, annotation ашиглахад хялбар	Test хийхэд хэцүү, framework-ээс хэт хамааралтай

1.1.5 Дүгнэлт

Dependency Injection-ийг хэрэгжүүлэх 3 үндсэн арга байдгаас constructor injection нь хамгийн найдвартай, тестлэхэд хялбар арга гэж үзэгддэг. Setter injection нь optional dependency-д тохиромжтой бол field injection нь annotation-д суурилсан framework-үүдэд өргөн хэрэглэгддэг. Энэхүү судалгааны ажлын хүрээнд annotation болон reflection механизмд тулгуурласан dependency injection framework хэрэгжүүлэх тул field injection арга нь онцгой ач холбогдолтой юм.

1.2 Annotation болон Reflection механизмын судалгаа

1.2.1 Java annotation-ийн үндсэн ойлголт

Annotation нь Java хэл дээр класс, интерфэйс, гишүүн функц, гишүүн өгөгдөл, параметр зэрэг программын элементүүдэд нэмэлт мэдээлэл буюу metadata өгөх зориулалттай хэлний механизм юм. Annotation нь өөрөө бизнес логик хэрэгжүүлдэггүй боловч компилятор, хөгжүүлэлтийн хэрэгсэл, framework болон runtime орчинд тухайн элементийн зориулалт, төлөв, хэрэглээний талаар мэдээлэл өгдөг.³

Java annotation-уудыг зориулалт, хэрэглээнээс нь хамааруулан built-in annotation, meta-annotation, custom annotation гэж ангилан авч үздэг.

Built-in annotation

Built-in annotation нь Java хэл өөрөө агуулдаг annotation-ууд бөгөөд ихэвчлэн компилятор болон хөгжүүлэгчид зориулсан байдаг. Эдгээр нь framework-ийн түвшний annotation биш, харин Java compiler-д нэмэлт заавар, тайлбар өгөх зорилготой.

Түгээмэл built-in annotation-уудад дараах annotation-ууд орно:

- `@Override`
- `@Deprecated`
- `@SuppressWarnings`
- `@FunctionalInterface`

Жишээлбэл, `@Override` annotation нь эцэг классаас өвлөсөн функцийг дахин тодорхойлж байгааг илэрхийлдэг бол `@Deprecated` annotation нь тухайн элемент хуучирсан, цаашид хэрэглэхээс зайлсхийх шаардлагатайг тэмдэглэдэг.

³Oracle. (2024). *Lesson: Annotations*. The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/java/annotations/>

Meta-annotation

Meta-annotation нь өөр annotation тодорхойлоход ашиглагддаг annotation юм. Өөрөөр хэлбэл annotation дээр хэрэглэгддэг annotation гэж үзэж болно. Java хэлэнд custom annotation боловсруулах үед meta-annotation-ууд чухал үүрэгтэй байдаг. Энэхүү дипломын ажлын хүрээнд хэрэгжүүлэх framework-д custom annotation үүсгэхдээ эдгээр meta-annotation-уудыг ашиглах шаардлагатай.

Түгээмэл meta-annotation-уудыг Хүснэгт 1.2 -д үзүүлэв.

Table 1.2: Java хэлний түгээмэл meta-annotation-ууд тэдгээрийн үүрэг

Annotation	Үүрэг
@Target	Annotation-ийг ямар төрлийн элемент дээр ашиглаж болохыг тодорхойлно
@Retention	Annotation ямар хугацаанд хадгалагдахыг тодорхойлно
@Documented	Annotation нь JavaDoc баримт бичигт харагдах эсэхийг заана
@Inherited	Annotation нь удамшлын үед subclass руу дамжих эсэхийг тодорхойлно
@Repeatable	Нэг annotation-ийг олон удаа хэрэглэх боломж олгоно

Custom annotation

Custom annotation нь meta-annotation ашиглан хэрэглэгч өөрийн шаардлагад нийцүүлэн тодорхойлсон annotation юм. Framework боловсруулах үед component, service, repository, inject зэрэг зориулалтаар custom annotation үүсгэх нь түгээмэл байдаг. Ийм annotation-ууд нь runtime үед reflection ашиглан уншигдаж, тухайн класст ямар үйлдэл хийхийг тодорхойлоход ашиглагдана.⁴

Жишээлбэл, дараах @Component annotation нь class түвшинд хэрэглэгдэх бөгөөд runtime үед хадгалагдахаар тодорхойлогдсон байна.

```
1 @Target(ElementType.TYPE)
2 @Retention(RetentionPolicy.RUNTIME)
3 public @interface Component {
4 }
```

Код 1.4: Custom annotation жишээ

1.2.2 Annotation-ий бүтцийн ангилал

Бүтцийн хувьд annotation-ийг дараах үндсэн төрлүүдэд ангилж үздэг.

1. **Marker annotation** – parameter агуулаагүй annotation.
2. **Single-value annotation** – нэг утга хүлээн авдаг annotation.
3. **Full annotation** – олон attribute буюу олон утга тодорхойлох боломжтой annotation.

⁴Oracle. (2024). *Annotations* (Java SE 17 & JDK 17). <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Annotation.html>

Marker annotation

Marker annotation нь ямар нэг parameter агуулаагүй, зөвхөн тухайн элементийн төлөв эсвэл зориулалтыг тэмдэглэх зориулалттай байдаг.

```
1 @Override
2 public String toString() {
3     return "Example";
4 }
```

Код 1.5: Marker annotation жишээ

Single-value annotation

Single-value annotation нь нэг үндсэн параметр авдаг.

```
1 @SuppressWarnings("unchecked")
2 public void process() {
3 }
```

Код 1.6: Single-value annotation жишээ

Full annotation

Full annotation нь олон attribute агуулж болдог тул илүү дэлгэрэнгүй тохиргоо өгөх боломжтой.

```
1 @RequestMapping(value="/users", method=RequestMethod.GET)
2 public List<User> getUsers() {
3     return users;
4 }
```

Код 1.7: Full annotation жишээ

1.2.3 Custom annotation үүсгэхэд ашиглагддаг үндсэн annotation-ууд

@Target

@Target annotation нь тухайн annotation-ийг ямар төрлийн элемент дээр хэрэглэхийг тодорхойлдог. Өөрөөр хэлбэл annotation-ийн хэрэглэгдэх хүрээг зааж өгдөг.

Table 1.3: @Target annotation-д ашиглагддаг ElementType болон тайлбарууд

ElementType	Тайлбар
TYPE	Class, interface, enum зэрэг төрөл дээр
FIELD	Классын талбар дээр
METHOD	Метод дээр
PARAMETER	Методын параметр дээр
CONSTRUCTOR	Constructor дээр
LOCAL_VARIABLE	Local variable дээр
ANNOTATION_TYPE	Өөр annotation дээр
PACKAGE	Package дээр

Жишээлбэл, дараах annotation нь зөвхөн class болон interface зэрэг төрөл дээр хэрэглэгдэхээр тодорхойлогдсон байна.⁵

```
1 @Target( ElementType .TYPE)
2 public @interface Component {
3 }
```

Код 1.8: @Target ашигласан жишээ

⁵Oracle. (2024). *Annotation Type Target*. Java SE 17 API Documentation. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Target.html>

@Retention

@Retention annotation нь тухайн annotation хэзээ хүртэл хадгалагдахыг тодорхойлдог. Энэ нь annotation-ийн амьдралын хугацааг заадаг бөгөөд compile-time болон runtime орчинд annotation ашиглах эсэхийг шийддэг.⁶

Retention policy нь дараах 3 төрөлтэй.

- SOURCE – Annotation нь зөвхөн source code түвшинд оршиж, compile хийх үед .class файл руу орохгүй устдаг.
- CLASS – Annotation нь compile хийсний дараа .class файлд хадгалагдах боловч runtime үед reflection-оор уншигдахгүй.
- RUNTIME – Annotation нь runtime үед хадгалагдаж, reflection ашиглан унших боломжтой.

Эдгээрийг Хүснэгт 1.4 -д харуулав.

Table 1.4: Retention policy-ийн орчны ялгаа

Policy	Source code	Class file	Runtime
SOURCE	✓	×	×
CLASS	✓	✓	×
RUNTIME	✓	✓	✓

⁶Oracle. (2024). *Annotation Type Target*. Java SE 17 API Documentation. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Retention.html>

Framework боловсруулах үед хамгийн чухал нь RUNTIME policy юм. Учир нь runtime үед reflection ашиглан annotation-ийг уншиж, dependency injection болон component scan зэрэг механизмыг хэрэгжүүлэх шаардлагатай байдаг.

Жишээ нь:

```
1 @Target( ElementType .TYPE)
2 @Retention( RetentionPolicy .RUNTIME)
3 public @interface Component {
4 }
```

Код 1.9: @Retention ашигласан жишээ

@Documented

@Documented annotation нь тухайн custom annotation JavaDoc баримт бичигт харагдах эсэхийг тодорхойлдог. Хэрэв annotation нь албан ёсны API баримт бичигт харагдах шаардлагатай бол энэ annotation-ийг хэрэглэнэ.

```
1 @Documented
2 @Target( ElementType .TYPE)
3 @Retention( RetentionPolicy .RUNTIME)
4 public @interface Service {
5 }
```

Код 1.10: @Documented ашигласан жишээ

@Inherited

@Inherited annotation нь class түвшний annotation удамшлын дагуу subclass-д дамжих эсэхийг тодорхойлдог. Энэ нь зөвхөн class дээр хэрэглэгдсэн annotation-д үйлчилдэг.

```
1 @Inherited
2 @Target( ElementType .TYPE)
3 @Retention( RetentionPolicy .RUNTIME)
```

```

4 public @interface Component {
5 }

```

Код 1.11: @Inherited ашигласан жишээ

@Repeatable

@Repeatable annotation нь нэг төрлийн annotation-ийг нэг элемент дээр олон удаа хэрэглэх боломж олгодог. Энэ нь ижил төрлийн олон metadata хадгалах шаардлагатай үед хэрэгтэй байдаг.

```

1 @Repeatable(Roles.class)
2 public @interface Role {
3     String value();
4 }

```

Код 1.12: @Repeatable ашигласан жишээ

1.2.4 *Spring Framework дахь annotation*

Spring Framework нь annotation-д суурилсан хөгжүүлэлтийн загварыг өргөн ашигладаг бөгөөд annotation-уудыг зориулалтаас нь хамааран component annotation, configuration annotation, dependency injection annotation, web annotation, AOP annotation, transaction annotation зэрэг ангиллаар авч үздэг.⁷

Component annotation

Component annotation нь Spring container-т bean болгох class-ыг тодорхойлоход ашиглагдана.

Жишээ нь:

- `@Component` – Package scanning үед framework энэ annotation-той class-уудыг илрүүлнэ. Энэхүү annotation-ийг IoC Container-д “Энэ классыг чи удирдаарай, объектыг нь үүсгээрэй” гэж хэлэхэд ашиглана. Тухайн annotation байгаа классыг IoC Container-т бүртгэхэд ашиглана.
- `@Service` – Business logic давхаргын class тэмдэглэх бөгөөд `@Component`-ийн тусгай хэлбэр юм.
- `@Repository` – Өгөгдлийн сангийн давхаргын class тодорхойлоход ашиглана.
- `@Controller` – Application хэсэгт end-point тэмдэглэх, илэрхийлэх annotation юм.

⁷Spring Framework Documentation. (2024). *Annotation-based Container Configuration*. <https://docs.spring.io/spring-framework/reference/core/beans/annotation-config.html>

Configuration annotation

Configuration annotation нь container-т тохиргоог үүсгэж тодорхойлж өгөхөд ашиглагддаг.

Жишээ нь:

- `@Configuration` – Тохиргооны класс тэмдэглэнэ. Bean method-уудыг агуулна. `@Component`-ийн тусгай хэлбэр учир container-т бүртгэгддэг.
- `@ComponentScan` – Хаана package scan хийхийг заана. value, basePackage буюу заасан package-с уншина.
- `@Bean` – Annotation-тай үүсгэх объектуудыг тодорхойлно.
- `@Import` – Өөр тохиргооны классыг оруулахад ашиглана. Том project-т тохиргоог хэд хэдэн class-т хуваан `@Import`-оор нэгтгэнэ.
- `@Scope` – Bean-ийн буюу тухайн annotation байгаа объектын амьдралын мөчлөгийг тодорхойлно.

Dependency Injection annotation

Dependency Injection annotation нь dependency injection буюу хамаарлыг илэрхийлэхэд ашиглагддаг.

Жишээ нь:

- `@Autowired` – Field болон constructor алинд inject хийх вэ гэдгийг тодорхойлно. Spring-д ажилладаг.
- `@Inject` – Dependency injection хийх field эсвэл constructor заана. Ямар ч JSR-330 container-т ажиллана.
- `@Qualifier` – Нэг interface-ийг хэд хэдэн class implement хийвэл аль нэгийг нь сонгон inject хийхэд ашиглана. Аль нэгийг нь `@Qualifier("stripeGateway")` гэж сонгож өгөх боломжтой.

- @Primary – @Qualifier байхгүй бол түүний implementation хийх өгөгдлийг энэ annotation-ийг тэмдэглэн илэрхийлж өгдөг.

Жишээ:

```

1 @Component("mysqlRepo")
2 public class MySQLUserRepository implements UserRepository {}
3
4 @Component("mongoRepo")
5 public class MongoUserRepository implements UserRepository {}
6
7 @Autowired
8 @Qualifier("mysqlRepo") // альнэгийгнъзаана
9 private UserRepository userRepo;
```

Код 1.13: @Qualifier ашигласан жишээ

```

1 @Primary // @Qualifier байхгүйүедэнэсонгодоно
2 public class StripePaymentGateway implements PaymentGateway { ... }
3
4 @Service
5 public class CheckoutService {
6     @Autowired
7     private PaymentGateway gateway; // -> StripePaymentGateway сонгогдоно
8 }
```

Код 1.14: @Primary ашигласан жишээ

Web annotation

Web annotation нь веб аппликэйшин хийхэд ашиглагддаг.

Жишээ нь:

- `@RequestMapping` – HTTP route тодорхойлох үндсэн annotation. Бүх HTTP method-д ажиллана (GET, POST, PUT, DELETE гэх мэт).
- `@GetMapping` – GET хүсэлтэд тодорхойлох annotation.
- `@PostMapping` – POST хүсэлтэд тодорхойлох annotation.
- `@DeleteMapping` – DELETE хүсэлтэд тодорхойлох annotation.
- `@UpdateMapping` – UPDATE хүсэлтэд тодорхойлох annotation.
- `@RestController` – `@Controller` + `@ResponseBody` хосолсон annotation. Бүх method-ийн буцаах утгыг JSON болгон хөрвүүлнэ.

AOP annotation

AOP annotation нь огтлолцсон логик буюу cross-cutting concern-уудыг хэрэгжүүлэхэд ашиглагдана.

Жишээ нь:

- `@Aspect`
- `@Before`
- `@After`
- `@Around`

Transaction annotation

Transaction annotation нь transaction удирдахад ашиглагдана.

Жишээ нь:

- `@Transactional`

1.2.5 Framework-д хэрэгжүүлэх annotation-ууд

Дипломд заавал хэрэгжүүлэх annotation-ууд (DI Framework-ийн цөм)

- @Component, @Service, @Repository – Bean бүртгэлт
- @Autowired, @Qualifier, @Primary – Dependency injection
- @Scope – Bean амьдралын мөчлөг

Хэрэгжүүлэх боломжтой annotation-ууд (нэмэлт)

- @GetMapping, @PostMapping – Энгийн HTTP routing
- @Configuration, @ComponentScan, @Bean, – Тохиргоо
- @Transactional – Энгийн transaction
- @Before, @After – Энгийн AOP logging

1.2.6 Java ба Spring annotation-ийн ялгаа

Java хэл дээр annotation нь программын элементүүдэд metadata нэмэх зориулалттай бөгөөд built-in annotation, meta-annotation болон custom annotation зэрэг төрлүүдэд ангилагддаг. Харин Spring Framework дээр annotation-ийг хэрэглээний зориулалтаар нь component annotation, configuration annotation, dependency injection annotation болон web annotation зэрэг ангиллаар ашигладаг. Энэхүү судалгааны ажилд annotation болон reflection механизмд тулгуурлан бүрэлдэхүүн хэсгүүдийг автоматаар илрүүлэх dependency injection framework-ийг хэрэгжүүлсэн.

1.2.7 Compile болон Runtime ойлголт

Compile гэдэг нь Java эх кодыг `javac` хөрвүүлэгч ашиглан `.class` файл болгон хөрвүүлэх үйл явц юм. Энэ үед `compiler` нь `syntax` алдааг шалгаж, Java Virtual Machine (JVM)-д ойлгогдох `bytecode` үүсгэдэг.

Runtime гэдэг нь хөрвүүлэгдсэн `.class` файлыг JVM санах ойд ачаалж, программыг ажиллуулах үе шат юм. Reflection механизм нь runtime үед ажилладаг бөгөөд `annotation` унших, `class`-ийн бүтэц шалгах, объект үүсгэх, метод дуудах зэрэг үйлдлүүдийг энэ шатанд гүйцэтгэдэг.

Иймээс `annotation`-г хэрэгжүүлэхийн тулд `annotation`-үүдийг `RetentionPolicy.RUNTIME` тохиргоотой тодорхойлох шаардлагатай байдаг.

1.2.8 Reflection механизм

Reflection нь Java хэлний runtime орчинд `class`, `method`, `field`, `constructor` зэрэг бүтцийн мэдээллийг шалгах, унших, өөрчлөх боломж олгодог механизм юм. Энэ механизм нь `annotation`-д суурилсан `framework`-ийн дотоод ажиллагаанд чухал үүрэгтэй.

Reflection ашигласнаар дараах боломжууд бүрдэнэ:

- `class`-ийн мэдээлэл унших,
- `field` болон `method` илрүүлэх,
- `annotation` байгаа эсэхийг шалгах,
- `constructor` ашиглан объект үүсгэх,
- `field`-д утга оноох,
- `method`-ийг runtime үед дуудах

Энэхүү дипломын ажлын хэрэгжүүлэлтэд reflection механизмыг ашиглан `custom annotation`-уудтай `class`-уудыг илрүүлэх, объект үүсгэх, `dependency`-үүдийг автоматаар холбох үндсэн

ажиллагааг хэрэгжүүлнэ.

1.2.9 Annotation ба Reflection-ийн уялдаа холбоо

Annotation болон reflection нь framework-ийн дотоод механизмд салшгүй холбоотой ажилладаг. Annotation нь класст metadata өгдөг бол reflection нь runtime үед уг metadata-г уншиж, түүнд үндэслэн шаардлагатай үйлдлийг гүйцэтгэдэг.

Жишээлбэл, хэрэв класс @Component annotation-той бол framework нь reflection ашиглан уг annotation-ийг илрүүлж, тухайн классыг container-д бүртгэж, шаардлагатай үед объект үүсгэх боломжтой болно. Үүнтэй адил @Inject annotation тэмдэглэгдсэн талбарыг reflection ашиглан олж, холбогдох dependency-г автоматаар оноож болно.

Иймээс annotation нь “юу хийх” талаар мэдээлэл өгдөг бол reflection нь “түүнийг хэрхэн хэрэгжүүлэх” механизмыг хангадаг гэж үзэж болно.

1.3 Ижил төрлийн framework-үүдийн судалгаа

Dependency Injection механизмыг хэрэгжүүлсэн түгээмэл framework-үүдэд Spring Framework, Google Guice, Jakarta CDI зэрэг орно. Эдгээр framework-үүд нь объектын хамаарлыг удирдах, container ашиглан объект үүсгэх, configuration боловсруулах зэрэг нийтлэг үүрэгтэй боловч хэрэгжүүлэлтийн арга барил, цар хүрээ, өргөтгөх боломжийн хувьд ялгаатай.

1.3.1 *Spring Framework*

Spring Framework нь Java экосистем дэх хамгийн өргөн хэрэглэгддэг framework-үүдийн нэг бөгөөд dependency injection, aspect-oriented programming, transaction management, web development зэрэг олон боломжийг агуулдагаараа онцлогтой. Энэ framework нь annotation-д суурилсан тохиргоо болон IoC container ашиглан объектуудын удирдлагыг төвлөрсөн байдлаар зохион байгуулдаг.



Зураг 1.3: Spring Framework

1.3.2 Google Guice

Google Guice нь dependency injection-д чиглэсэн, харьцангуй энгийн бүтэцтэй Java framework юм. Энэ нь Spring-тэй харьцуулахад боломжийн хүрээ бага боловч DI зарчмыг ойлгох, жижиг болон дунд хэмжээний системд хэрэгжүүлэхэд тохиромжтой.⁸



Зураг 1.4: Google Guice

1.3.3 Jakarta CDI

Jakarta CDI (Contexts and Dependency Injection) нь enterprise Java орчинд dependency injection болон component lifecycle management хэрэгжүүлэх стандарт тодорхойлолт юм. Энэ нь Java enterprise орчны албан ёсны стандартуудын нэг хэсэг бөгөөд enterprise түвшний системд өргөн ашиглагддаг.



Зураг 1.5: Jakarta CDI

⁸Google. (2024). *Google Guice: A lightweight dependency injection framework for Java*. GitHub. <https://github.com/google/guice/wiki/Motivation>

1.3.4 Framework-үүдийн харьцуулалт

Table 1.5: Түгээмэл DI framework-үүдийн харьцуулалт

Framework	Үндсэн зориулалт	Онцлог
Spring Framework	Иж бүрэн application framework	IoC, DI, annotation, web, data access зэрэг өргөн боломжтой
Google Guice	DI чиглэлийн framework	Энгийн бүтэцтэй, dependency injection-д төвлөрсөн
Jakarta CDI	Java enterprise стандарт DI	Enterprise орчинд component lifecycle болон injection дэмждэг

Хүснэгт 1.5-оос харахад Spring Framework нь хамгийн өргөн боломжтой, харин Google Guice нь илүү төвлөрсөн DI шийдэл болох нь ажиглагдана.

1.4 Ашиглагдах программчлалын хэлний судалгаа

Энэхүү дипломын ажлын хэрэгжүүлэлтэд Java программчлалын хэлийг ашигласан. Java хэл нь объект хандалтат загварчлалыг бүрэн дэмждэг, өргөн хөгжүүлэлтийн экосистемтэй, annotation болон reflection механизмыг хэлний түвшинд агуулдаг тул судалгааны сэдэвтэй шууд нийцэж байна.

Java хэлний үндсэн давуу талд:

- объект хандалтат бүтэц,
- annotation болон reflection дэмжлэг,
- олон платформ дээр ажиллах чадвар,
- framework хөгжүүлэхэд тохиромжтой сангууд,
- өргөн хэрэглээ, баримтжуулалт

зэрэг орно.



Зураг 1.6: Java программчлалын хэл

1.5 Ашиглагдах технологи, хэрэгслүүдийн судалгаа

1.5.1 *Maven*

Maven нь Java төслийн dependency management болон build management хийх хэрэгсэл юм. Энэхүү ажлын хүрээнд төслийн сангуудыг удирдах, бүтэц зохион байгуулалтыг стандартчилах, хэрэгжүүлэлтийг эмх цэгцтэй байлгах зорилгоор ашиглана.⁹



Зураг 1.7: Maven builder

1.5.2 *JUnit*

JUnit нь Java орчинд unit test бичих түгээмэл хэрэгсэл юм. Хэрэгжүүлсэн framework-ийн үндсэн функцүүд болох annotation илрүүлэлт, объект үүсгэх, dependency injection хийх ажиллагааг шалгахын тулд JUnit ашиглах боломжтой.



Зураг 1.8: JUnit test

⁹Apache Software Foundation. (2024). *Maven – Welcome to Apache Maven*. <https://maven.apache.org/index.html>

1.5.3 Хөгжүүлэлтийн орчин

Хөгжүүлэлтийн орчноор IntelliJ IDEA эсвэл Eclipse зэрэг Java хөгжүүлэлтийг дэмждэг integrated development environment ашиглаж болно. Эдгээр орчин нь код бичих, dependency удирдах, debug хийх, тест ажиллуулах зэрэг олон талын боломж олгодог.¹⁰



Зураг 1.9: IntelliJ IDEA



Зураг 1.10: Eclipse

1.5.4 Git болон GitHub

Git нь хувилбар удирдлагын систем бөгөөд төслийн хөгжүүлэлтийн явц дахь өөрчлөлтүүдийг хянахад ашиглагдана. GitHub нь тухайн repository-г хадгалах, хувилбаруудыг удирдах, хөгжүүлэлтийн явцыг баримтжуулахад дэмжлэг үзүүлнэ.



Зураг 1.11: Git GitHub

¹⁰JetBrains. (2024). *IntelliJ IDEA: The Leading Java and Kotlin IDE*. <https://www.jetbrains.com/idea/>

1.6 Системийн ерөнхий шийдлийн судалгаа

Энэхүү дипломын ажлын хүрээнд хэрэгжүүлэх framework нь annotation болон reflection механизмд тулгуурласан суурь түвшний dependency injection framework юм. Иймээс шийдлийн судалгааны хэсэгт ижил зарчмаар ажилладаг Spring Framework-ийн IoC container, bean удирдлага, configuration, dependency injection болон bean scope-ийн ойлголтуудыг авч үзэх нь зүйтэй. Spring нь enterprise түвшний иж бүрэн framework боловч түүний үндсэн IoC/DI зарчим нь энэхүү ажлын хэрэгжүүлэлтийн логиктой шууд холбоотой.

1.6.1 Spring Framework-ийн IoC container

Spring framework-ийн IoC container нь объектыг үүсгэх, тохируулах, dependency-г шийдэх үндсэн үүргийг гүйцэтгэдэг. Энэ container-ийн суурь ойлголтууд хэрэгжүүлэлтүүд нь BeanFactory болон ApplicationContext юм.¹¹

BeanFactory

BeanFactory нь IoC container-ийн хамгийн суурь түвшний root interface бөгөөд bean объектыг үүсгэх, хадгалах, буцаах үндсэн үүрэгтэй. Үүний онцлог нь bean-ийг ихэвчлэн `getBean()` дуудагдсан үед үүсгэдэгт оршино. Иймээс санах ойн хэрэглээг хэмнэх боломжтой боловч орчин үеийн Spring төслүүдэд дангаар нь түгээмэл ашигладаггүй.

```
1 BeanFactory factory =  
2     new XmlBeanFactory(new ClassPathResource("beans.xml"));  
3  
4 UserService service = factory.getBean(UserService.class);
```

Код 1.15: BeanFactory ашигласан жишээ

¹¹Spring Framework Documentation. (2024). *The IoC Container*. <https://docs.spring.io/spring-framework/reference/core/beans.html>

ApplicationContext

ApplicationContext нь BeanFactory-ийг өргөтгөсөн интерфейс бөгөөд bean удирдлагаас гадна event system, resource management, annotation support зэрэг нэмэлт боломжуудыг олгодог. Орчин үеийн Spring төслүүдэд ихэвчлэн ApplicationContext ашиглагддаг.

```
1 ApplicationContext context =  
2     new ClassPathXmlApplicationContext("applicationContext.xml");  
3  
4 UserService service = context.getBean(UserService.class);
```

Код 1.16: ApplicationContext ашигласан жишээ

Иймээс энэхүү дипломын ажлын хүрээнд хэрэгжүүлэх DI framework-ийн container ойлголтыг тодорхойлохдоо ApplicationContext-ийн ажиллагааг жишиг болгон авч үзэх боломжтой.

1.6.2 Bean болон container-д бүртгэгдэх классууд

IoC container-аар удирдагдаж буй объектыг *bean* гэж нэрлэдэг. Spring-д зөвхөн тодорхой annotation-оор тэмдэглэгдсэн класс эсвэл @Bean annotation-оор буцаасан объект bean болж бүртгэгдэнэ.

Bean болдог түгээмэл annotation-ууд:

- @Component
- @Service
- @Repository
- @Controller
- @Bean

Жишээ нь:


```

1 @Component
2 public class UserRepositoryImpl implements UserRepository {
3     public String findUser(int id) {
4         return "Хэрэглэгч:" + id;
5     }
6 }
7
8 @Service
9 public class UserService {
10     private final UserRepository repo;
11
12     @Autowired
13     public UserService(UserRepository repo) {
14         this.repo = repo;
15     }
16 }

```

Код 1.17: Component болон Service annotation ашигласан жишээ

Харин дараах annotation-ууд нь bean бүртгэхгүй:

- @Override
- @Deprecated
- @SuppressWarnings
- @Autowired
- @Getter, @Setter

Өөрөөр хэлбэл зарим annotation нь зөвхөн metadata өгдөг бол зарим нь container-ийн бодит ажиллагаанд оролцдог.

1.6.3 Spring configuration хийх үндсэн аргууд

Spring framework-д container-д ямар объект бүртгэх, dependency-г хэрхэн шийдэхийг configuration-оор тодорхойлдог. Үүнийг хэрэгжүүлэх хэд хэдэн үндсэн арга бий.

XML суурилсан configuration

Spring-ийн анхны арга нь bean бүрийг XML файлд тодорхойлох байсан.

```
1 <beans xmlns="http://www.springframework.org/schema/beans">
2
3     <bean id="userRepository"
4         class="com.myapp.UserRepositoryImpl"/>
5
6     <bean id="userService"
7         class="com.myapp.UserService">
8         <constructor-arg ref="userRepository"/>
9     </bean>
10
11 </beans>
```

Код 1.18: XML суурилсан configuration жишээ

```
1 ApplicationContext context =
2     new ClassPathXmlApplicationContext("applicationContext.xml");
3
4 UserService service = context.getBean(UserService.class);
```

Код 1.19: XML суурилсан configuration жишээ

Энэ арга нь ойлгомжтой боловч тохиргоо ихсэх тусам XML файл төвөгтэй болдог.

Annotation-д суурилсан configuration

Энэ аргад bean болон dependency-г класс дээр annotation ашиглан тодорхойлно.

```
1 @Component
2 public class UserRepositoryImpl implements UserRepository {
3 }
4
5 @Service
6 public class UserService {
7     private final UserRepository repo;
8
9     @Autowired
10    public UserService(UserRepository repo) {
11        this.repo = repo;
12    }
13 }
```

Код 1.20: Annotation-д суурилсан bean бүртгэлтийн жишээ

```
1 <beans>
2     <context:component-scan base-package="com.myapp"/>
3 </beans>
```

Код 1.21: Component scan тохиргооны жишээ

Энэ нь XML-ийг багасгаж, код болон тохиргоог ойртуулсан давуу талтай.

Java-д суурилсан configuration

Орчин үед хамгийн түгээмэл хэрэглэгддэг арга нь Java configuration юм.

```
1 @Configuration
2 @ComponentScan("com.myapp")
3 public class AppConfig {
4
5     @Bean
6     public UserRepository userRepository() {
7         return new UserRepositoryImpl();
8     }
9
10    @Bean
11    public UserService userService() {
12        return new UserService(userRepository());
13    }
14 }
```

Код 1.22: Java-д суурилсан configuration жишээ

```
1 ApplicationContext context =
2     new AnnotationConfigApplicationContext(AppConfig.class);
3
4 UserService service = context.getBean(UserService.class);
```

Код 1.23: Java configuration-оос container үүсгэх жишээ

Энэ арга нь XML-ийг бараг бүрэн орлуулсан бөгөөд кодоор configuration удирдах боломж олгодог.

Spring Boot configuration

Spring Boot нь дээрх тохиргоонуудыг илүү автоматжуулсан хэлбэрээр хэрэгжүүлдэг.

```
1 @SpringBootApplication
```

```

2 public class Main {
3     public static void main(String[] args) {
4         SpringApplication.run(Main.class, args);
5     }
6 }

```

Код 1.24: Spring Boot configuration жишээ

`@SpringBootApplication` нь `@Configuration`, `@ComponentScan` гэсэн ойлголтуудыг нэгтгэн агуулдаг.¹²

1.6.4 *Dependency Injection ба annotation ашиглалт*

Dependency Injection нь объект өөрийн dependency-г шууд үүсгэхийн оронд container-аас авч ашиглах зарчим юм. Spring-д үүнийг annotation-оор хялбар илэрхийлдэг.

```

1 @Service
2 public class UserService {
3
4     private final UserRepository repo;
5
6     @Autowired
7     public UserService(UserRepository repo) {
8         this.repo = repo;
9     }
10 }

```

Код 1.25: Autowired ашигласан dependency injection жишээ

Энд `@Autowired` annotation нь `UserRepository` төрлийн bean-ийг автоматаар оноож өгнө. Энэ зарчим нь классуудын хамаарлыг сулруулж, турших болон өргөтгөхөд хялбар болгодог.

¹²Spring Framework Documentation. (2024). *Java-based Container Configuration*. <https://docs.spring.io/spring-framework/reference/core/beans/java.html>

1.6.5 Bean scope

Bean scope нь тухайн bean хэдэн хувь үүсэх, ямар хугацаанд оршин байхыг тодорхойлдог. Spring-д хэд хэдэн төрөл байдаг боловч энэхүү ажлын хүрээнд дараах хоёр scope хамгийн чухал гэж үзэж болно.

Singleton scope

Singleton нь bean зөвхөн нэг удаа үүсэж, container-ийн хүрээнд дахин дахин ижил объектыг буцаана.

```
1 @Component
2 @Scope("singleton")
3 public class AppConfigManager {
4 }
```

Код 1.26: Singleton scope ашигласан жишээ

Энэ нь төлөв хадгалдаггүй service төрлийн классуудад тохиромжтой.

Prototype scope

Prototype нь bean дуудагдах бүрд шинэ объект үүсгэнэ.

```
1 @Component
2 @Scope("prototype")
3 public class TempDataHolder {
4 }
```

Код 1.27: Prototype scope ашигласан жишээ

Энэ нь объект бүр өөрийн төлөвтэй байх шаардлагатай үед илүү тохиромжтой.

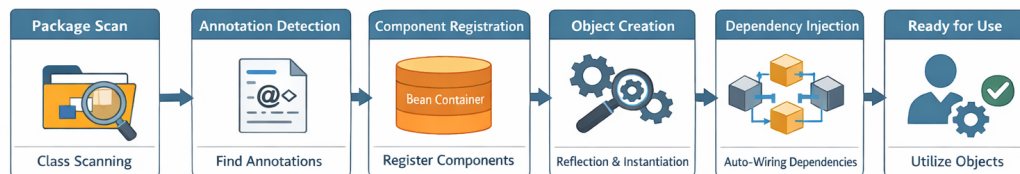
1.6.6 Хэрэгжүүлэх framework-ийн ерөнхий шийдэлтэй уялдах нь

Дээрх судалгаанаас үзэхэд Spring Framework-ийн IoC container нь annotation-оор классуудыг илрүүлэх, bean болгон бүртгэх, dependency-г автоматаар шийдэх, объектын амьдралын мөчлөгийг удирдах нэгдсэн механизмыг хэрэгжүүлдэг. Энэхүү дипломын ажлын хүрээнд хэрэгжүүлэх framework нь Spring-ийн бүх боломжийг бүрэн хуулбарлахгүй боловч түүний үндсэн IoC/DI зарчмыг хялбаршуулсан түвшинд хэрэгжүүлэхэд чиглэнэ.

Тодруулбал хэрэгжүүлэх framework-ийн ажиллагааг дараах үе шатуудад авч үзэж болно:

1. package эсвэл тодорхой классуудыг scan хийх,
2. annotation-уудыг илрүүлэх,
3. component классуудыг container-д бүртгэх,
4. reflection ашиглан объект үүсгэх,
5. dependency-г автоматаар оноох,
6. хэрэглээний түвшинд бэлэн объектыг ашиглах

Иймээс Spring Framework-ийн дээрх шийдлийн судалгаа нь энэхүү ажлын онолын суурь болохын зэрэгцээ хэрэгжүүлэх dependency injection framework-ийн бүтэц, үндсэн боломжуудыг тодорхойлох жишиг загвар болж байна.



Зураг 1.12: Хэрэгжүүлэх DI Framework-ийн ерөнхий ажиллагааны зураглал

Зураг 1.12-т хэрэгжүүлэх framework-ийн ерөнхий урсгал, үндсэн бүрэлдэхүүн хэсгүүдийн хамаарлыг харуулсан.

1.7 Бүлгийн дүгнэлт

Энэ бүлэгт энэхүү дипломын ажлын хэрэгжүүлэлтийн онолын болон технологийн суурь нөхцөлүүдийг судлав. Үүнд framework-ийн ерөнхий ойлголт, IoC болон DI зарчим, annotation болон reflection механизм, ижил төрлийн framework-үүдийн онцлог, мөн хэрэгжүүлэлтэд ашиглагдах Java хэл, Maven, JUnit, Git зэрэг технологи, хэрэгслүүдийг авч үзлээ. Судалгааны үр дүнд annotation болон reflection-д суурилсан суурь түвшний DI Framework хэрэгжүүлэхэд Java орчин хамгийн тохиромжтой бөгөөд Spring Framework нь судалгааны жишиг болгон ашиглагдах үндэслэлтэй болох нь тогтоогдож байна.

2. СИСТЕМИЙН ШААРДЛАГА

Энэхүү бүлэгт annotation болон reflection механизмд суурилсан dependency injection framework-ийн шаардлагыг тодорхойлно. Уг framework нь component классуудыг автоматаар илрүүлэх, container-д бүртгэх, dependency-г шийдэх, bean-ийн lifecycle-ийг удирдах суурь түвшний боломжуудыг хэрэгжүүлэхэд чиглэнэ. Иймээс энэ бүлэгт framework-ийн функциональ болон функциональ бус шаардлагууд, хэрэглээний үндсэн үйл явц, боломжит классууд, CRC карт, классын диаграммыг тус тус тодорхойлно.

2.1 Framework-ийн шаардлагын тодорхойлолт

2.1.1 Ерөнхий шаардлага

Хөгжүүлэх framework нь Java хэл дээр хэрэгжих бөгөөд annotation болон reflection механизмд тулгуурлан dependency injection-ийн суурь үйл ажиллагааг хэрэгжүүлнэ. Framework-ийн үндсэн зорилго нь хэрэглэгчийн тодорхойлсон классуудыг package түвшинд илрүүлэх, annotation-уудыг унших, bean болгон бүртгэх, объект үүсгэх, dependency-г автоматаар оноох, улмаар хэрэглэгчид бэлэн объектыг container-оос авах боломж олгоход оршино.

Энэ зорилгын хүрээнд framework нь дараах үндсэн боломжуудыг дэмжсэн байна:

- custom annotation-уудыг runtime үед унших,
- package scanning хийх,
- bean бүртгэх,
- singleton болон prototype төлөвийг удирдах,
- dependency injection гүйцэтгэх,
- bean-ийг нэр болон төрлөөр хайж авах,

- алдааг тодорхой exception-оор боловсруулах.

2.1.2 Функциональ шаардлага

Функциональ шаардлага нь framework-ийн заавал хийж гүйцэтгэх үндсэн үйлдлүүдийг тодорхойлно. Энэхүү framework нь сургалтын болон судалгааны зориулалттай суурь DI container тул Spring Framework-ийн бүх боломжийг бус, харин dependency injection-ийн гол цөм механизмуудыг хэрэгжүүлэхэд чиглэнэ.

Table 2.1: Framework-ийн функциональ шаардлага

ID	Шаардлагын нэр	Тодорхойлолт
ФШ-10	Package scanning	Заасан package болон түүний дэд package-уудаас класс илрүүлэх боломжтой байна.
ФШ-11	Annotation detection	@Component, @Autowired, @Qualifier, @Scope annotation-уудыг runtime үед илрүүлдэг байна.
ФШ-12	Bean registration	@Component тэмдэглэгээтэй классуудыг container-д bean хэлбэрээр бүртгэдэг байна.
ФШ-13	Object instantiation	Reflection API ашиглан bean-ийн объектыг автоматаар үүсгэдэг байна.
ФШ-14	Dependency injection	Constructor болон field түвшний dependency injection дэмждэг байна.
ФШ-15	Bean retrieval	Bean-ийг нэрээр болон төрлөөр getBean() аргаар авах боломжтой байна.
ФШ-16	Scope management	Singleton болон prototype scope-ийг дэмждэг байна.
ФШ-17	Exception handling	Bean үүсгэх, dependency шийдэх үед гарсан алдааг тодорхой exception-оор боловсруулдаг байна.

2.2 Функциональ бус шаардлага

Функциональ бус шаардлага нь framework-ийн чанар, гүйцэтгэл, ашиглахад хялбар байдал, тестлэгдэх чадвар зэрэг шинжүүдийг тодорхойлно. Энэхүү framework нь enterprise түвшний иж бүрэн платформ биш боловч судалгааны ажилд ашиглагдах тул тодорхой чанарын шалгууртай байх ёстой.

Table 2.2: Framework-ийн функциональ бус шаардлагууд

ID	Ангилал	Шаардлага	Хэмжүүр / Тодорхойлолт
ФБШ-10	Гүйцэтгэл	Startup хурд	10 хүртэл bean-тэй үед container-ийн эхлэлт 100ms-аас бага байна.
ФБШ-11	Гүйцэтгэл	Bean авах хурд	getBean() дуудлага 1ms-аас бага хугацаанд хариулахуйц байна.
ФБШ-12	Найдвартай байдал	Circular dependency илрүүлэлт	Тойрог хамаарал илэрсэн тохиолдолд startup үед тодорхой алдаа өгч зогсоно.
ФБШ-13	Санах ойн удирдлага	Singleton cache	Singleton bean-ийг давтан үүсгэхгүй, cache бүтэц ашиглан хадгална.
ФБШ-14	Ашиглахад хялбар байдал	API энгийн байдал	ApplicationContext үүсгэхэд зөвхөн package нэр өгөхөд хангалттай байна.
ФБШ-15	Суурилуулалт	Гуравдагч талын хамааралгүй байдал	Framework нь зөвхөн Java SE орчинд, нэмэлт library-гүй ажиллана.
ФБШ-16	Тестлэгдэх чадвар	Unit тест	Үндсэн функциональ шаардлага бүр JUnit 5 тестээр баталгаажсан байна.
ФБШ-17	Тайлбарлагдах чадвар	Лог гаргалт	Startup үед бүртгэгдсэн болон inject хийгдсэн bean-үүдийн мэдээллийг харуулдаг байна.

Эдгээр шаардлагууд нь framework-ийг зөвхөн ажилладаг байхаас гадна ойлгомжтой, туршигдахуйц, өргөтгөх боломжтой байдлаар хэрэгжүүлэхэд чиглэнэ.

2.3 Custom annotation-уудын шаардлага

2.3.1 *Meta-annotation ашиглах шаардлага*

Custom annotation тодорхойлоход дараах meta-annotation-ууд чухал үүрэгтэй.

- `@Target` – annotation ямар төрлийн программын элемент дээр хэрэглэгдэхийг заана;
- `@Retention` – annotation ямар хугацаанд хадгалагдахыг тодорхойлно;

Энэхүү framework-ийн хувьд хамгийн чухал нь `@Target` болон `@Retention` юм. Ялангуяа `@Retention(RetentionPolicy.RUNTIME)` нь annotation-ийг программ ажиллах үед Reflection API ашиглан унших боломжийг бүрдүүлдэг. Хэрэв annotation нь runtime үед хадгалагдахгүй бол container тухайн классыг bean мөн эсэх, эсвэл ямар dependency inject хийх шаардлагатайг илрүүлэх боломжгүй болно. Иймээс framework-д ашиглагдах бүх үндсэн annotation-ууд заавал `RUNTIME retention`-тэй байна.

Table 2.3: Framework-д хэрэгжүүлэх custom annotation-уудын шаардлага

Annotation	Target	Retention	Зорилго
@Component	TYPE	RUNTIME	Тухайн классыг container-г bean болгон бүртгэхийг заана. Нэрийг value() атрибутаар өгч болно.
@Autowired	FIELD, CONSTRUCTOR	RUNTIME	Dependency-г автоматаар inject хийхийг тэмдэглэнэ. Bean олдохгүй үед алдаа боловсруулах боломжтой.
@Qualifier	FIELD, PARAMETER	RUNTIME	Ижил төрлийн олон bean байвал яг аль bean-ийг inject хийхийг нэрээр заана.
@Scope	TYPE	RUNTIME	Bean-ийн амьдралын мөчлөгийг тодорхойлно. Энэ ажлын хүрээнд singleton болон prototype төрлийг дэмжинэ.

2.3.2 Retention policy болон runtime reflection-ийн шаардлага

Дэвшүүлсэн асуулт: Annotation-д суурилсан framework хэрэгжүүлэх үед яагаад заавал RUNTIME түвшинд хадгалагдаж, reflection ашиглан унших боломжтой байх шаардлагатай байдаг вэ?

Хариулт: @Retention нь annotation ямар хугацаанд хадгалагдахыг тодорхойлдог бөгөөд энэ нь тухайн annotation-ийг программын аль үе шатанд ашиглахыг шийддэг. Өөрөөр хэлбэл annotation зөвхөн эх кодын түвшинд хэрэгтэй юу, компайл хийгдсэн класс файлд хадгалагдах уу, эсвэл программ ажиллаж байх үед runtime орчинд уншигдах шаардлагатай юу гэдгийг

ялгаж өгдөг.

Ийм ангилал хийсэн гол шалтгаан нь бүх annotation ижил зориулалттай биш байдагтай холбоотой. Зарим annotation зөвхөн компиляторт зориулсан байдаг бол зарим нь хөгжүүлэлтийн хэрэгсэл, баримтжуулалт, эсвэл runtime үед ажилладаг framework-д зориулсан байдаг. Тиймээс annotation-ийг ашиглах түвшнээс нь хамааруулан ялгаж өгсөн нь Java хэлний уян хатан, зорилгод нийцсэн шийдэл юм.

Энэхүү судалгааны ажлын хувьд RUNTIME policy хамгийн чухал байр суурь эзэлж байна. Учир нь annotation-д суурилсан DI framework нь программ ажиллаж эхэлсний дараа л классуудыг илрүүлэх, тэдгээрийн annotation-ийг шалгах, объект үүсгэх, хамаарлыг холбох зэрэг үйлдлүүдийг гүйцэтгэдэг. Энэ бүх процесс compile үе шатанд биш, runtime үед явагддаг. Иймээс framework annotation-уудыг reflection-оор уншихын тулд заавал RetentionPolicy.RUNTIME тохиргоотой байх шаардлагатай.

Runtime үед reflection ашиглах шаардлагыг дараах үндсэн гурван талаас тайлбарлаж болно.

Динамик илрүүлэлт

Framework нь ерөнхий зориулалттай байх ёстой. Өөрөөр хэлбэл framework-ийг ашиглах хөгжүүлэгч ямар классууд үүсгэх, ямар dependency нэмэх, системээ хэрхэн өргөжүүлэхийг framework урьдчилан мэдэх боломжгүй. Иймээс framework нь хэрэглэгчийн шинээр тодорхойлсон классуудыг программ ажиллах мөчид динамикаар илрүүлдэг байх шаардлагатай.

Программ эхлэх үед framework нь classpath-ийг нэгжиж, тодорхой annotation бүхий классуудыг олж илрүүлнэ. Энэ нь reflection-ийн хамгийн чухал давуу тал болох динамик чанарыг илэрхийлдэг. Жишээлбэл, хөгжүүлэгч @MyComponent annotation бүхий шинэ класс нэмсэн тохиолдолд framework үүнийг compile үе шатанд бус, runtime үед л бодитоор илрүүлж чадна.

```
1 Class<?> clazz = Class.forName("com.myapp.UserService");  
2 Object obj = clazz.getDeclaredConstructor().newInstance();
```

Код 2.1: Reflection ашиглан классыг динамикаар ачаалах жишээ

Энэ нь framework-д уян хатан байдал олгож, хөгжүүлэгчийн тодорхойлсон шинэ бүрэлдэхүүнүүдийг урьдчилан хатуу кодлолгүйгээр ажиллуулах боломж бүрдүүлдэг.

Fail-fast буюу эрт баталгаажуулалт

Runtime үед reflection ашиглахын өөр нэг чухал шалтгаан нь framework системийн тохиргоо, хамаарал, объектын бүтэц зэрэгт алдаа байгаа эсэхийг эрт илрүүлэх боломжтой байдагт оршино. Программ compile хийгдэж .class файл үүссэний дараа JVM дээр ажиллах үед framework annotation-уудыг уншиж, dependency-үүдийг шалгаж, шаардлагатай объектуудыг үүсгэнэ.

Энэ үе шатанд хэрэв шаардлагатай dependency олдохгүй, хоёрдмол сонголт үүссэн, эсвэл буруу тохиргоо хийсэн бол framework шууд алдаа өгч зогсох боломжтой. Үүнийг *fail-fast* зарчим гэж үздэг. Энэ нь программыг хожуу шатанд гацах, тодорхойгүй алдаа гаргах эрсдэлийг бууруулж, тохиргооны алдааг эхэнд нь илрүүлэх ач холбогдолтой.

Өөрөөр хэлбэл compile үе шат нь зөвхөн синтакс болон төрөл шалгах үндсэн баталгаажуулалтыг хийдэг бол runtime үе шатанд framework нь архитектурын түвшний баталгаажуулалт хийж байна гэж ойлгож болно. Тиймээс reflection-ийг runtime үед ашиглах нь зөвхөн объект үүсгэх хэрэгсэл биш, мөн dependency graph-ийн зөв байдлыг шалгах механизм болдог.

Уламжлалт объект үүсгэлтээс ялгарах нь

Java хэл дээр объектыг ерөнхийдөө хоёр аргаар үүсгэж болно.

1. **Static object creation** – Хөгжүүлэгч классын нэрийг урьдчилан мэдэж, new түлхүүр үгээр шууд объект үүсгэнэ.
2. **Dynamic object creation** – Классын нэрийг текст хэлбэрээр эсвэл runtime мэдээлэлд үндэслэн авч, reflection ашиглан объект үүсгэнэ.

Жишээлбэл:

```
1 UserService service1 = new UserService(); // static
```

```
2  
3 Class<?> clazz = Class.forName("com.myapp.UserService");  
4 Object service2 = clazz.getDeclaredConstructor().newInstance(); // dynamic
```

Код 2.2: Static ба dynamic object creation-ийн ялгаа

2.3.3 *ApplicationContext-ийн шаардлага*

ApplicationContext нь framework-ийн төв бүрэлдэхүүн хэсэг бөгөөд IoC container-ийн үүргийг гүйцэтгэнэ. Энэ бүрэлдэхүүн хэсэг нь annotation-уудыг унших, bean-үүдийн мета мэдээллийг хадгалах, объект үүсгэх, dependency inject хийх, мөн хэрэглэгчид bean буцаах бүх үндсэн логикийг удирдана. Иймээс framework-ийн хамгийн чухал шаардлагууд ApplicationContext-т төвлөрнө.

ApplicationContext-ийн гүйцэтгэх үндсэн үүрэг

ApplicationContext нь дараах үндсэн үйлдлүүдийг дэмжсэн байх шаардлагатай.

1. **Эхлүүлэлт** – хэрэглэгч ApplicationContext(String packageName) хэлбэрээр package нэр дамжуулахад тухайн package болон дэд package-ууд дахь классуудыг автоматаар илрүүлэх;
2. **Bean бүртгэл** – @Component annotation-той классуудыг bean гэж тодорхойлон, тэдгээрийн мета мэдээллийг BeanDefinition хэлбэрээр бүртгэх;
3. **Object үүсгэлт** – singleton scope-той bean-үүдийг нэг удаа үүсгэж cache-д хадгалах;
4. **Dependency injection** – @Autowired annotation-той талбар бүрийн хамаарлыг container-с хайж, Reflection ашиглан inject хийх;
5. **Bean retrieval API** – getBean(Class<T>) болон getBean(String) хэлбэрээр bean буцаах;
6. **Алдаа боловсруулалт** – bean олдохгүй байх, олон тохирох bean илрэх, circular dependency үүсэх зэрэг тохиолдолд тодорхой exception шидэх.

Bean бүртгэл ба мета мэдээлэл

Container нь runtime үед илрүүлсэн bean бүрийн тухай дараах мэдээллийг хадгалах шаардлагатай.

- bean-ийн нэр,
- тухайн bean-д харгалзах класс,
- scope-ийн төрөл,
- annotation metadata,
- singleton тохиолдолд үүссэн instance.

Эдгээрийг нэг дор хадгалах үүднээс BeanDefinition хэмээх тусгай бүтэц ашиглах нь оновчтой. BeanDefinition нь bean-ийн бодит объект биш, харин тухайн bean-ийг хэрхэн үүсгэх, ямар шинжтэй болохыг илэрхийлэх мета мэдээллийн объект байна. Ингэснээр container дотоод логигоороо уян хатан ажиллах боломжтой болно.

Dependency injection-ийн шаардлага

Dependency injection-ийн хувьд container нь эхлээд шаардлагатай bean-ийг type-аар хайж олох, шаардлагатай тохиолдолд @Qualifier утгаар илүү нарийвчлан сонгох, дараа нь Reflection API ашиглан талбарт утга оноох логиктой байна. Хэрэв bean огт олдохгүй бол NoSuchBeanException, ижил төрлийн олон bean тохирвол тусгай алдаа, харин $A \rightarrow B \rightarrow A$ хэлбэрийн тойрог хамаарал үүсвэл CircularDependencyException шидэх шаардлагатай.

Энэ нь fail-fast зарчмыг хэрэгжүүлж буй хэлбэр юм. Өөрөөр хэлбэл framework нь run-time эхлэлийн үеэр dependency-үүдийн алдааг аль болох эрт илрүүлж, тодорхой мэдэгдэл өгөх ёстой. Энэ шаардлага нь сургалтын болон туршилтын зорилготой framework-ийн хувьд онцгой ач холбогдолтой.

Scope удирдлагын шаардлага

Container нь дор хаяж дараах хоёр төрлийн scope-ийг дэмжинэ.

- **Singleton scope** – нэг bean нэг л удаа үүсэж, дараагийн бүх хүсэлтэд ижил instance буцаагдана;

- **Prototype scope** – bean хүсэх бүрд шинэ instance үүсгэнэ.

Singleton scope нь container-ийн cache механизмыг шаарддаг бөгөөд энэ нь bean авах хурдыг нэмэгдүүлнэ. Prototype scope нь төлөвтэй объектуудыг тус тусад нь үүсгэх боломж олгодог. Энэхүү хоёр scope нь DI framework-ийн үндсэн lifecycle удирдлагыг харуулахад хангалттай гэж үзэв.

ApplicationContext сонгосон үндэслэл

Spring-ийн архитектурт BeanFactory болон ApplicationContext гэсэн хоёр гол түвшин байдаг. BeanFactory нь хамгийн суурь container бол ApplicationContext нь үүнийг өргөтгөсөн, илүү өндөр түвшний боломжуудтай хувилбар юм. ApplicationContext гэсэн нэршлийг ашиглах нь оновчтой гэж үзэв. Учир нь уг нэршил нь IoC container, bean lifecycle, dependency resolution гэсэн ойлголтуудыг нэгдмэл байдлаар илэрхийлж чадна. Мөн сургалтын түвшинд Spring Framework-тэй харьцуулан тайлбарлахад илүү ойлгомжтой.

2.3.4 Технологийн шаардлага

Framework-ийн хэрэгжилт нь аль болох гуравдагч талын сангаас хамааралгүй, зөвхөн Java SE стандарт сангууд дээр тулгуурласан байх шаардлагатай. Энэ нь framework-ийн дотоод механизмыг илүү тодорхой харуулах бөгөөд Reflection, class loading, annotation inspection зэрэг гол ойлголтуудыг гаднын abstraction-гүйгээр судлах боломж олгоно. Үүнтэй холбоотойгоор дараах технологийн орчныг сонгов.

Table 2.4: Framework хэрэгжүүлэх технологийн шаардлага

Ангилал	Хэрэгсэл / Технологи	Тайлбар
Программчлалын хэл	Java SE 17 (LTS)	Annotation, Reflection, class loading зэрэг механизмыг стандарт түвшинд дэмжинэ.
Build tool	Apache Maven 3.9.x	Төслийн бүтэц, compile, dependency болон test-ийг удирдахад ашиглана.
IDE	IntelliJ IDEA Community Edition	Reflection-д суурилсан кодын debug болон runtime төлөв ажиглахад тохиромжтой.
Тестийн орчин	JUnit 5	Framework-ийн үндсэн үйлдлүүдэд unit тест бичиж баталгаажуулахад ашиглана.
JVM	OpenJDK 17 / Temurin 17	Java 17 орчныг тогтвортой ажиллуулах виртуал машин.
Нэмэлт сан	Байхгүй	Зөвхөн JDK стандарт сан ашиглана.

Java 17 сонгосон үндэслэл

Java 17 нь урт хугацааны дэмжлэгтэй (LTS) хувилбар бөгөөд орчин үеийн Java хөгжүүлэлтэд тогтвортой суурь орчин гэж үзэгддэг. Annotation, Reflection, generics, module system зэрэг

энэхүү судалгааны ажилд шаардлагатай боломжуудыг бүрэн дэмждэг. Мөн Java 17 ашигласнаар framework-ийн хэрэгжилт орчин үеийн JVM орчинд туршигдаж, цаашид өргөтгөхөд илүү найдвартай суурь бүрдэнэ.

IntelliJ IDEA сонгосон үндэслэл

Энэхүү framework-ийн хэрэгжилт нь runtime үед Reflection API ашиглан класс илрүүлэх, annotation шалгах, private field-д хандах, объект үүсгэх зэрэг олон шаттай далд логик агуулна. Иймээс хөгжүүлэлтийн орчин нь зөвхөн код бичих хэрэгсэл байхаас гадна санах ой дахь объектын төлөвийг ажиглах, алхамчилсан debug хийх, stack trace болон exception-ийн урсгалыг шинжлэх боломжтой байх ёстой.

Энэ шаардлагын үүднээс IntelliJ IDEA нь илүү оновчтой орчин гэж үзэв. Учир нь уг IDE нь Java төслүүдийн бүтцийг сайн таньдаг, Maven-тэй нягт уялддаг, мөн Reflection-д суурилсан логикийг алхам алхмаар мөрдөж шалгахад тохиромжтой. Харин Eclipse болон VS Code нь тодорхой нөхцөлд ашиглаж болох боловч runtime inspection болон deep debugging-ийн хувьд IntelliJ IDEA илүү давуу гэж үзэж байна.

Гуравдагч талын сан ашиглахгүй байх шаардлага

Энэ ажлын нэг чухал зорилго нь DI framework-ийн үндсэн ажиллагааг “бэлэн сан” ашиглалгүйгээр өөрийн түвшинд хэрэгжүүлж үзэх явдал юм. Хэрэв classpath scanning, dependency injection, bean registry зэрэг механизмыг гуравдагч талын сангаар шийдвэл судалгааны ажлын гол шинж болох дотоод механизмын ойлголт бүдгэрэх эрсдэлтэй. Тиймээс зөвхөн JDK стандарт сан ашиглах нь энэхүү ажлын судалгааны үнэ цэнийг нэмэгдүүлнэ.

Технологийн шаардлагын дүгнэлт

Ийнхүү framework-ийн технологийн орчныг сонгохдоо дараах гурван шалгуурыг баримталсан.

- **тогтвортой байдал** – Java 17 LTS, OpenJDK орчин;

- **хөгжүүлэлтийн хялбар байдал** – Maven бүтэц, JUnit тестийн дэмжлэг;
- **судалгааны ил тод байдал** – гуравдагч талын сангуй, цэвэр Java SE хэрэгжилт.

Эдгээр шаардлагуудыг хангаснаар framework-ийн дараагийн шат болох архитектурын зохиомж, модулийн хуваарилалт, container-ийн дотоод урсгалыг илүү оновчтой тодорхойлох боломж бүрдэнэ.

2.4 Статик загвар

2.4.1 Бүрэлдэхүүн хэсгүүдийн тодорхойлолт

Framework дараах үндсэн бүрэлдэхүүн хэсгүүдээс тогтоно:

- `ApplicationContext` – container-ийн үндсэн удирдлага
- `BeanDefinition` – bean-ийн мета мэдээлэл
- `ClassPathScanner` – classpath scanning хийх модуль
- `DependencyInjector` – dependency injection хийх модуль
- `BeanFactory` – bean үүсгэх логик
- custom annotation-ууд – `@Component`, `@Autowired`, `@Qualifier`, `@Scope`
- exception классууд – runtime алдаа боловсруулах хэсэг

2.4.2 Боломжит классууд

- **ApplicationContext**
 - Шинж: bean registry, singleton cache, scanner, injector
 - Үүрэг: container эхлүүлэх, bean бүртгэх, bean буцаах

- **BeanDefinition**

- Шинж: `beanName`, `beanClass`, `scope`
- Үүрэг: `bean`-ийн мета мэдээллийг хадгалах

- **ClassPathScanner**

- Шинж: `basePackage`
- Үүрэг: `package scan` хийж `component class` илрүүлэх

- **DependencyInjector**

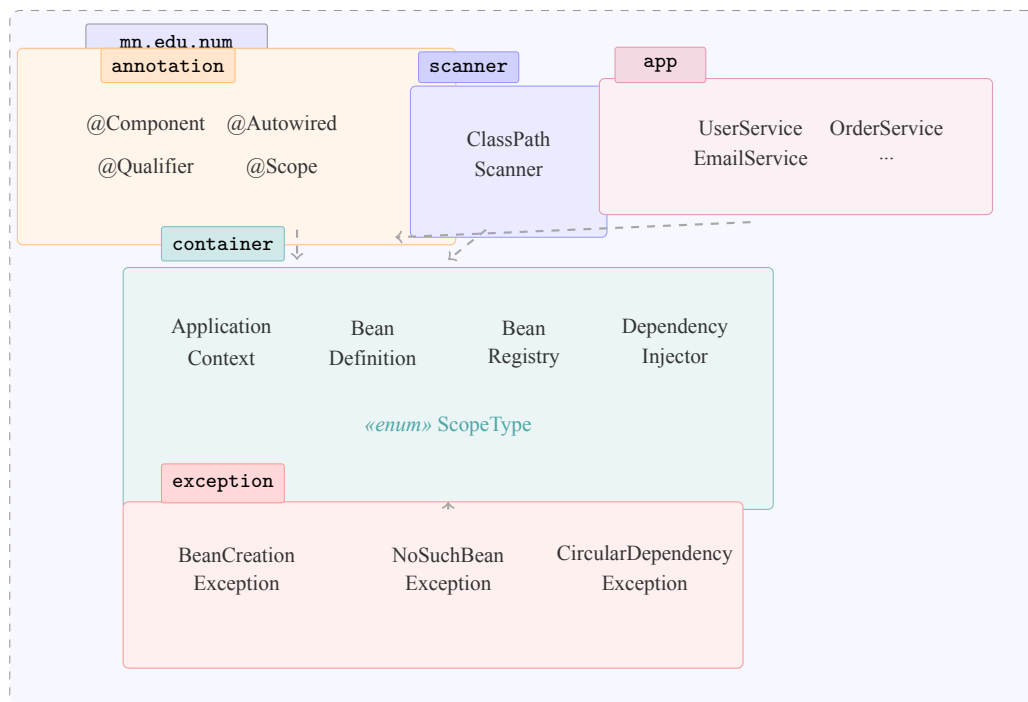
- Үүрэг: `@Autowired`, `@Qualifier` annotation-уудыг уншиж `dependency` оноох

- **ScopeType**

- Утга: `SINGLETON`, `PROTOTYPE`

2.4.3 Багцын зохион байгуулалт

Багц (*package*) нь загварын элементийг нэгтгэн хураангуйлдаг бүтцийн нэгж юм. Том хэмжээний загварыг илүү тоймтой, ойлгомжтой болгох, системийн бүтцийн өндөр хийсвэрлэлийн түвшний тайлбар өгөх зорилготой. Энэхүү `framework`-ийн хэрэгжүүлэлтийг бүхэлд нь `mn.edu.num` нэртэй нэг үндсэн багцад байрлуулсан бөгөөд дотор нь давхаргат архитектурын зарчмыг дагасан таван дэд багцаас бүрдэнэ.



Зураг 2.1: DI framework-ийн багцын диаграмм

Дээрх диаграммд харуулсан таван дэд багц бүр нь давхаргат архитектурын нэг давхаргад тус тус харгалзана.

annotation багц нь framework-ийн хэрэглэгчтэй харилцах гадаад интерфэйсийн үүрэг гүйцэтгэнэ. @Component, @Autowired, @Qualifier, @Scope гэсэн дөрвөн custom annotation агуулах бөгөөд бүгд @Retention(RetentionPolicy.RUNTIME) тохиргоотой байна.

scanner багц нь ClassPathScanner нэртэй нэг класс агуулна. Энэ класс нь заасан package болон дэд package-уудыг рекурсив аргаар нэгжиж @Component annotation-той классуудыг илрүүлдэг.

container багц нь framework-ийн цөм бүрэлдэхүүн хэсгүүдийг агуулна. ApplicationContext нь IoC container-ийн үндсэн удирдлага, BeanDefinition нь bean-ийн мета мэдээлэл, BeanRegistry нь бүртгэл болон singleton кэш, DependencyInjector нь dependency inject хийх логик, ScopeType нь bean-ийн амьдралын мөчлөгийн enum төрөл юм.

exception багц нь framework-ийн fail-fast зарчмыг хэрэгжүүлэх гурван тусгай exception

агуулна: BeanCreationException, NoSuchBeanException, CircularDependencyException.

app багц нь framework-ийг ашиглах жишээ хэрэглэгчийн классуудыг агуулах туршилтын зориулалттай багц юм. Энэ багц нь framework-ийн нэг хэсэг биш, харин хэрэглэгчийн кодыг дүрслэн харуулах зорилготой.

Багцуудын хоорондын харилцааг дараах хүснэгтэд тоймлов.

Table 2.5: Багцуудын хоорондын хамаарал

Хамааралтай багц	Ашиглах багц	Тайлбар
scanner	annotation	@Component, @Scope-г унших
scanner	container	BeanDefinition объект үүсгэж буцаах
container	annotation	@Autowired, @Qualifier-г унших
container	exception	Алдааны нөхцөлд exception шидэх
app	annotation	@Component, @Autowired тавих
app	container	ApplicationContext үүсгэж ашиглах

2.4.4 КҮХ карт (CRC card)

Table 2.6: ApplicationContext классын КҮХ карт

Responsibilities	Collaborations
Container-ийг эхлүүлэх	ClassPathScanner
Component class-уудыг scan хийх үйл явцыг удирдах	ClassPathScanner
BeanDefinition-уудыг бүртгэх	BeanDefinition
Singleton төрлийн bean-үүдийг cache-д хадгалах	ScopeType
Dependency injection үйл явцыг эхлүүлэх	DependencyInjector
Хүссэн bean-ийг нэр эсвэл төрлөөр нь буцаах	BeanDefinition, ScopeType
Bean үүсгэх ба lifecycle-ийг хянах	DependencyInjector

Table 2.7: BeanDefinition классын KYX карт

Responsibilities	Collaborations
Bean-ийн нэрийг хадгалах	ApplicationContext
Bean-ийн классын төрлийг хадгалах	ApplicationContext
Bean-ийн scope мэдээллийг хадгалах	ScopeType
Bean үүсгэхэд шаардлагатай мета мэдээлэл өгөх	ApplicationContext, DependencyInjector
Container-д бүртгэгдсэн bean-ийн тодорхойлолтыг төлөөлөх	ApplicationContext

Table 2.8: ClassPathScanner классын KYX карт

Responsibilities	Collaborations
Өгөгдсөн base package-ийг scan хийх	ApplicationContext
ClassPath доторх class-уудыг илрүүлэх	ApplicationContext
@Component зэрэг annotation бүхий class-уудыг таних	BeanDefinition
Component class-уудын мэдээллийг ApplicationContext-д дамжуулах	ApplicationContext
Scan хийх логикийг төвлөрүүлэн хэрэгжүүлэх	ApplicationContext

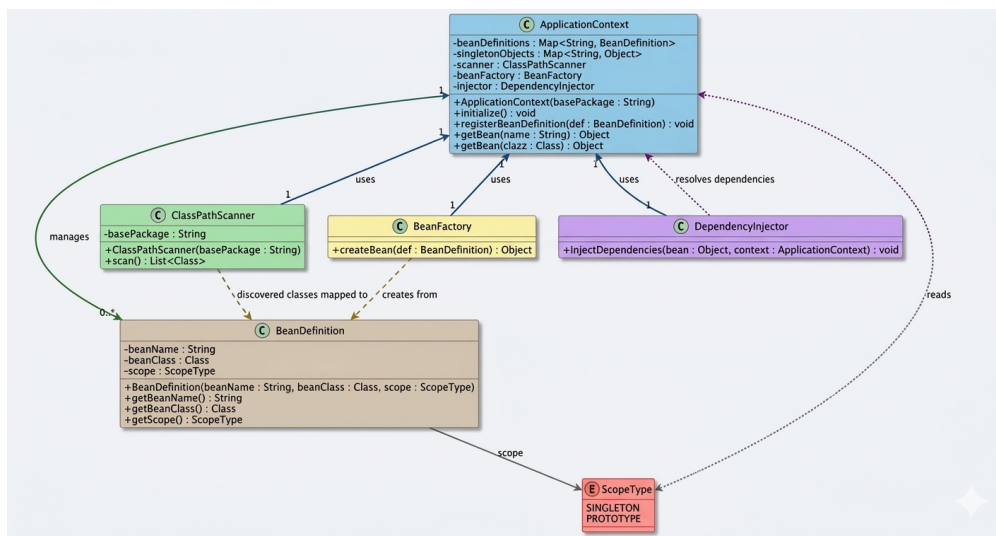
Table 2.9: DependencyInjector классын КҮХ карт

Responsibilities	Collaborations
@Autowired annotation-ийг унших	ApplicationContext
@Qualifier annotation-ийг унших	ApplicationContext
Bean-үүдийн хоорондын dependency-г тодорхойлох	BeanDefinition
Тохирох dependency-г олж inject хийх	ApplicationContext
Field эсвэл constructor injection хэрэгжүүлэх	BeanDefinition
Dependency байхгүй эсвэл зөрчилтэй үед алдаа мэдээлэх	ApplicationContext

Table 2.10: ScopeType төрлийн КҮХ карт

Responsibilities	Collaborations
Bean-ийн scope төрлийг тодорхойлох	BeanDefinition
SINGLETON болон PROTOTYPE утгыг илэрхийлэх	ApplicationContext
Bean үүсгэх болон дахин ашиглах логикт чиглэл өгөх	ApplicationContext
Container bean-ийг cache хийх эсэхийг шийдэхэд ашиглагдах	ApplicationContext

2.4.5 Классын диаграмм



Зураг 2.2: Annotation-д суурилсан DI framework-ийн классын диаграмм

2.2-р зурагт annotation-д суурилсан DI framework-ийн классын диаграммыг харуулав. Диаграммд **ApplicationContext**, **ClassPathScanner**, **BeanFactory**, **BeanDefinition**, **DependencyInjector**, **ScopeType** гэсэн зургаан үндсэн бүрэлдэхүүн хэсэг болон тэдгээрийн харилцан хамаарал тусгагдсан байна.

ApplicationContext нь framework-ийн төв класс бөгөөд **beanDefinitions**, **singletonObjects**, **scanner**, **beanFactory**, **injector** гэсэн таван хувийн талбар агуулна. Энэ класс нь **ClassPathScanner**, **BeanFactory**, **DependencyInjector** гурвыг *uses* харилцаагаар ашиглах бөгөөд **ClassPathScanner**-тэй *manages* харилцаагаар холбогдоно.

ClassPathScanner нь **basePackage** талбартай бөгөөд **scan()** аргаар илэрсэн классуудыг **BeanDefinition**-д харгалзуулан *discovered classes mapped to* харилцаагаар дамжуулна. **BeanFactory** нь **createBean()** аргаар **BeanDefinition**-д тулгуурлан **bean** объект үүсгэх *creates from* үүрэгтэй.

BeanDefinition нь **bean**-ийн мета мэдээллийг хадгалах **beanName**, **beanClass**, **scope** талбаруудтай бөгөөд **scope** талбар нь **ScopeType** enum-тэй *scope* харилцаагаар холбогдоно. **ScopeType** нь **SINGLETON** болон **PROTOTYPE** гэсэн хоёр утгыг агуулна. **DependencyInjector** нь **ApplicationContext**-

оос *resolves dependencies* болон *reads* харилцаагаар шаардлагатай bean-уудыг хайж inject хийнэ.

2.5 Динамик загвар

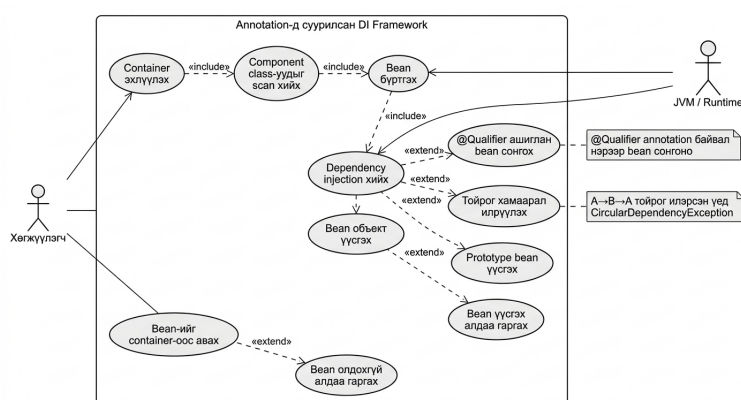
2.5.1 Framework хэрэглэгчийн ажлын явцын загвар (Use case)

Framework-ийн үндсэн actor нь хөгжүүлэгч байна.

- Систем – Annotation-д суурилсан DI Framework
- Actor – Хөгжүүлэгч

Хөгжүүлэгчийн framework-тэй харилцах үндсэн ажлын явцууд:

- Container эхлүүлэх
- Component class-уудыг scan хийх
- Bean бүртгэх
- Dependency injection хийх
- Bean-ийг container-оос авах



Зураг 2.3: Use case

2.3-р зурагт хөгжүүлэгч болон JVM actor - уудын ажлын явцын диаграммыг харуулсан.

2.5.2 Ажлын явцын задаргаа (Use case description)

Container эхлүүлэх

Table 2.11: Container эхлүүлэх use case-ийн тодорхойлолт

Ажлын явц	Container эхлүүлэх
Тайлбар	Хөгжүүлэгч framework-ийн ApplicationContext объектыг үүсгэж container-ийг эхлүүлнэ.
Actor	Хөгжүүлэгч
Урьдчилсан нөхцөл	Package нэр болон framework-ийн үндсэн тохиргоо өгөгдсөн байна.
Үндсэн алхам	1. Хөгжүүлэгч ApplicationContext үүсгэнэ. 2. Framework package scanning хийнэ. 3. @Component annotation-тай классуудыг илрүүлнэ. 4. BeanDefinition үүсгэж container-д бүртгэнэ. 5. Шаардлагатай singleton bean-үүдийг үүсгэнэ.
Үр дүн	Framework ашиглахад бэлэн container үүссэн байна.

Bean авах

Table 2.12: Bean авах use case-ийн тодорхойлолт

Ажлын явц	Bean авах
Тайлбар	Хөгжүүлэгч container-оос шаардлагатай bean-ийг нэрээр эсвэл төрлөөр авна.
Actor	Хөгжүүлэгч
Урьдчилсан нөхцөл	Container амжилттай эхэлсэн, bean бүртгэгдсэн байна.
Үндсэн алхам	1. Хөгжүүлэгч <code>getBean()</code> аргыг дуудна. 2. Framework bean definition-ийг шалгана. 3. Singleton бол хадгалсан instance буцаана. 4. Prototype бол шинэ instance үүсгэнэ.
Үр дүн	Хүссэн bean хөгжүүлэгчид буцаагдана.

2.5.3 Бүлгийн дүгнэлт

Энэхүү бүлэгт framework-ийн шаардлагыг тодорхойлов. Үүнд framework-ийн функциональ болон функциональ бус шаардлагууд, хөгжүүлэгчийн ашиглах үндсэн use case-ууд, мөн framework-ийг бүрдүүлэх үндсэн классууд, КҮХ карт, классын диаграммын бүтэц зэргийг системтэйгээр тодорхойлсон. Эдгээр нь дараагийн бүлэгт хэрэгжүүлэлт болон туршилтын хэсгийг боловсруулах суурь болно.

3. СИСТЕМИЙН ЗОХИОМЖ

3.1 Архитектурын ерөнхий үндэслэл

Энэхүү судалгааны ажлын хүрээнд хөгжүүлж буй annotation-д суурилсан dependency injection framework нь энгийн хэрэглээний программ бус, харин бусад программын ажиллагааг дэмжих суурь программ хангамжийн орчин буюу framework шинжтэй систем юм. Иймээс түүний архитектур нь зөвхөн модуль хуваарилалт төдий бус, харин объект илрүүлэлт, мета өгөгдөл уншилт, bean бүртгэл, объект үүсгэлт, хамаарал шийдэлт, алдаа боловсруулалт зэрэг дотоод механизмыг ойлгомжтой, өргөтгөх боломжтой, туршихад хялбар байдлаар зохион байгуулах шаардлагатай.

3.2 Сонгосон архитектур: Давхаргат архитектур

Энэхүү framework-ийн хэрэгжилтэд хамгийн тохиромжтой шийдэл нь давхаргат архитектур (Layered Architecture) гэж үзсэн. Учир нь уг судалгааны ажлын гол зорилго нь enterprise хэмжээний тархмал систем байгуулах бус, харин annotation болон reflection-д тулгуурласан IoC/DI механизмын дотоод ажиллагааг загварчлан хэрэгжүүлэхэд оршино.

Давхаргат архитектур нь системийн бүрэлдэхүүн хэсгүүдийн үүргийг тодорхой зааглаж, давхаргууд хоорондын хамаарлыг хязгаарласнаар өөрчлөлтийн нөлөөг багасгах, кодын уншигдах байдлыг сайжруулах, мөн тест хийх боломжийг нэмэгдүүлэх давуу талтай.¹

¹Microsoft, “Layered (N-tier) Architecture”, <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>

3.3 Сонгосон архитектурын давуу тал

Энэхүү framework-д layered architecture ашигласнаар дараах давуу талууд бий болно.

1. **Annotation тодорхойлолт болон runtime логикийн тусгаарлалт.** @Component, @Autowired зэрэг annotation-ууд нь зөвхөн мета мэдээлэл агуулж, тэдгээрийг боловсруулах логик нь container болон injector давхаргад төвлөрнө. Ингэснээр annotation-ийн тодорхойлолтыг өөрчлөхгүйгээр боловсруулах логикийг бие даан өргөтгөх боломж бүрдэнэ.
2. **Scanner давхаргын өргөтгөх чадвар.** Package scanning нь тусдаа давхарга болж байрласнаар ClassPathScanner-ийг цаашид JAR файл дэмжих, тусгай шүүлтүүр (*filter*) нэмэх зэрэг хувилбаруудаар өргөтгөх боломж бүрдэнэ.
3. **Мета өгөгдөл болон instance хадгалалтын тусгаарлалт.** BeanDefinition болон BeanRegistry нь тусдаа байснаар bean-ийн мета өгөгдөл болон runtime-д үүсгэгдсэн instance хадгалалтыг оновчтой зохион байгуулах, ирээдүйд бие даан сайжруулах боломж бүрдэнэ.
4. **Fail-fast зарчмын хэрэгжилт.** Exception давхаргыг тусгаарласнаар систем нь алдааг аль болох эрт, тодорхой мэдэгдлийн хамт илрүүлдэг *fail-fast* шинж чанартай болно. Энэ нь хөгжүүлэгчид алдааны шалтгааныг хурдан ойлгоход тустай.
5. **Spring Framework-ийн архитектурын суурь ойлголттой нийцэл.** Энэхүү архитектур нь Spring-ийн суурь ойлголттой нийцэж байгаа бөгөөд ApplicationContext нь bean-ийг үүсгэх, удирдах, dependency-г холбох үндсэн container болохыг баталдаг.²

²Spring Framework Documentation, “The IoC Container”, <https://docs.spring.io/spring-framework/docs/current/reference/html/core.html>

3.4 Layered Architecture Diagram

Энэхүү диаграмм нь системийн давхаргат бүтцийг харуулна.

- **Annotation layer**

- @Component
- @Autowired
- @Qualifier
- @Scope

- **Scanner layer**

- ClassPathScanner

- **Container layer**

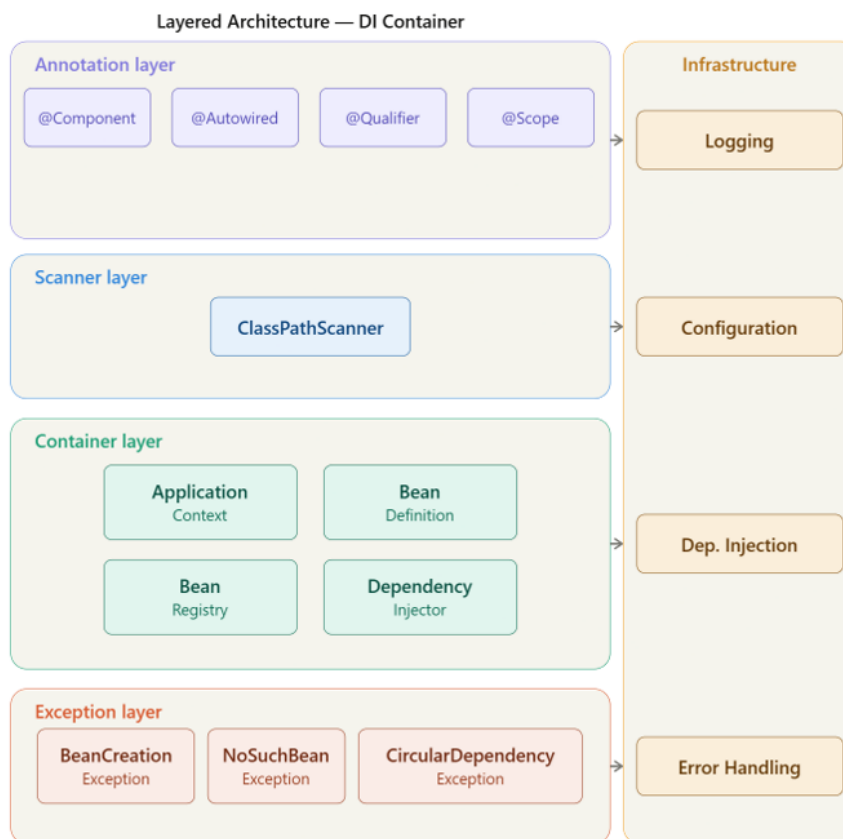
- ApplicationContext
- BeanDefinition
- BeanRegistry
- DependencyInjector

- **Exception layer**

- BeanCreationException
- NoSuchBeanException
- CircularDependencyException

Энэхүү диаграммыг Visual Paradigm Online tool ашиглан боловсруулсан болно.³

³Visual Paradigm Online, <https://online.visual-paradigm.com>



Зураг 3.1: Төслийн архитектур

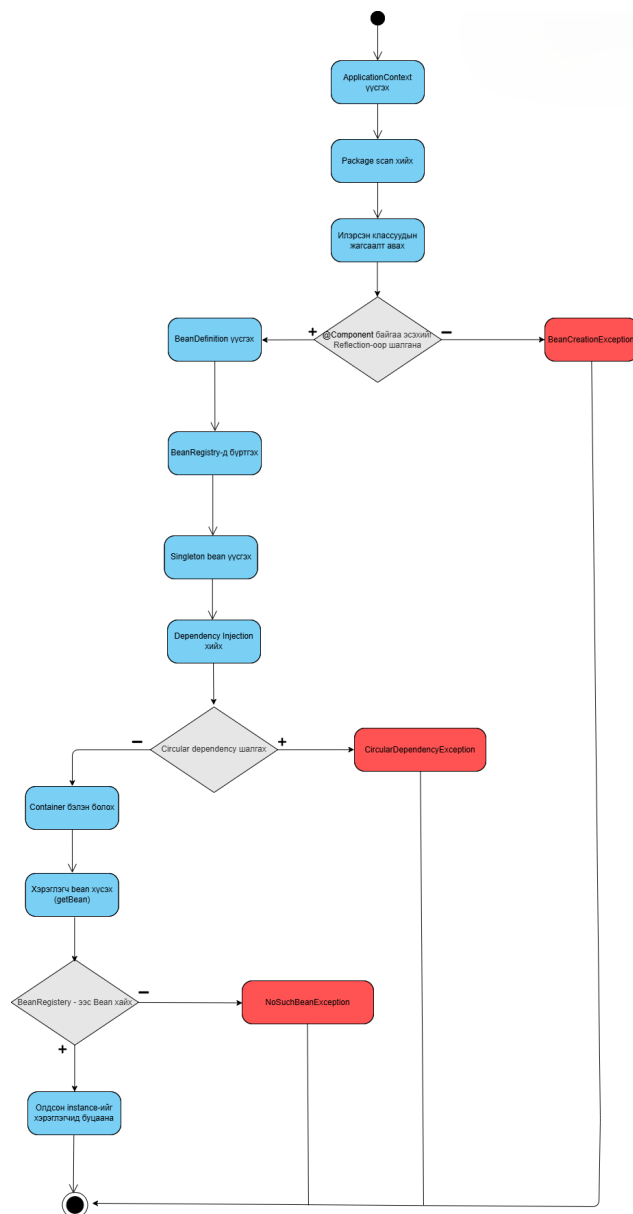
3.1-р зурагт annotation-д суурилсан DI framework-ийн давхаргат архитектурыг харуулав. Архитектур нь annotation давхарга, scanner давхарга, container давхарга, exception давхарга гэсэн дөрвөн үндсэн давхаргаас бүрдэх бөгөөд баруун талд infrastructure хэсэг болох logging, configuration, dependency injection, error handling үйлчилгээнүүд байрлана. Annotation давхарга нь @Component, @Autowired, @Qualifier, @Scope annotation-уудыг агуулж, scanner давхарга нь ClassPathScanner ашиглан классуудыг илрүүлнэ. Container давхарга нь ApplicationContext, BeanDefinition, BeanRegistry, DependencyInjector гэсэн дөрвөн үндсэн бүрэлдэхүүнээс тогтох бол exception давхарга нь BeanCreationException, NoSuchBeanException, CircularDependencyException гэсэн гурван exception классаас бүрдэнэ.

3.5 Activity Diagram

Activity diagram нь системийн startup болон runtime үеийн ажиллагааны дарааллыг илэрхийлнэ. Эхлээд хэрэглэгч ApplicationContext үүсгэж, ClassPathScanner ашиглан package scanning хийнэ. Үүний дараа илэрсэн class бүрийг шалган @Component annotation бүхий class-уудаас BeanDefinition үүсгэн BeanRegistry-д бүртгэнэ.

Дараагийн шатанд singleton bean instance-үүдийг үүсгэж, DependencyInjector ашиглан dependency injection гүйцэтгэнэ. Хэрэв object үүсгэх явцад алдаа гарвал BeanCreationException, харин dependency шийдвэрлэх үед тойрог хамаарал илэрвэл CircularDependencyException шидэгдэнэ. Эдгээр тохиолдолд процесс шууд зогсоно.

Initialization амжилттай дууссаны дараа container бэлэн болж, хэрэглэгч getBean() аргаар шаардлагатай bean-ийг авах боломжтой болно. Хэрэв хүссэн bean олдохгүй бол NoSuchBeanException шидэгдэнэ.



Зураг 3.2: Activity diagramm

3.2-р зурагт DI framework-ийн ажиллах дарааллын диаграммыг харуулав. Диаграмм нь ApplicationContext үүсгэхээс эхлэн хэрэглэгчид bean буцаах хүртэлх бүх үе шатыг дүрсэлнэ.

Эхний үе шатанд ApplicationContext үүсгэгдэж, package scan хийгдэн илэрсэн классуудын жагсаалт авагдана. Дараа нь @Component annotation байгаар Reflection API ашиглан шалгах бөгөөд annotation олдохгүй тохиолдолд BeanCreationException шидэгдэж процесс зогсоно.

Annotation олдсон тохиолдолд BeanDefinition үүсгэгдэж BeanRegistry-д бүртгэгдэнэ.

Бүртгэлтийн дараа singleton bean-ууд урьдчилан үүсгэгдэж dependency injection хийгдэнэ. Энэ үе шатанд circular dependency шалгалт явагдах бөгөөд тойрог хамаарал илэрсэн тохиолдолд CircularDependencyException шидэгдэнэ. Тойрог хамаарал байхгүй бол container бэлэн болж хэрэглэгч getBean() дуудах боломжтой болно. getBean() дуудагдахад BeanRegistry-ээс bean хайгдах бөгөөд bean олдохгүй тохиолдолд NoSuchBeanException шидэгдэнэ. Bean олдвол тухайн instance хэрэглэгчид буцаагдаж процесс амжилттай дуусна.

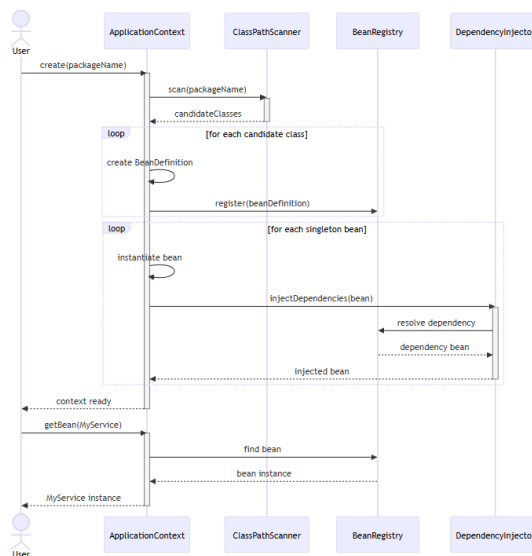
3.6 Sequence Diagram

Sequence diagram нь объектуудын хоорондын харилцан ажиллагааны дарааллыг илэрхийлнэ. Эхлээд хэрэглэгч ApplicationContext-ийг үүсгэж, package scanning эхлүүлнэ. ClassPathScanner нь боломжит class-уудыг илрүүлж буцаана.

Үүний дараа ApplicationContext нь @Component annotation бүхий class-уудаас BeanDefinition үүсгэн BeanRegistry-д бүртгэнэ. Дараа нь singleton bean-үүдийг үүсгэж, DependencyInjector ашиглан dependency injection гүйцэтгэнэ.

Dependency injection хийх явцад шаардлагатай dependency-г BeanRegistry-ээс хайж олно. Хэрэв dependency шийдвэрлэх үед тойрог хамаарал илэрвэл CircularDependencyException, харин bean үүсгэх явцад алдаа гарвал BeanCreationException үүснэ.

Initialization дууссаны дараа хэрэглэгч getBean() дуудахад ApplicationContext нь BeanRegistry-ээс шаардлагатай bean-ийг хайж олно. Хэрэв bean олдвол instance-ийг буцааж, эсрэг тохиолдолд NoSuchBeanException шидэгдэнэ.



Зураг 3.3: Sequence diagramm

3.3-р зурагт DI framework-ийн startup болон bean авах үйл явцын дарааллын диаграммыг харуулав. Диаграммд User, ApplicationContext, ClassPathScanner, BeanRegistry, DependencyInjector

гэсэн таван оролцогч харилцан үйлчилнэ.

Эхний үе шатанд хэрэглэгч `create(packageName)` дуудахад `ApplicationContext` нь `ClassPathScanner` ийг ажиллуулж `scan(packageName)` гүйцэтгэнэ. `Scanner` нь `candidateClasses` буцааж өгөх бөгөөд `ApplicationContext` нь `candidate` класс бүрийг давтан `BeanDefinition` үүсгэж `register(beanDefinition)` ашиглан `BeanRegistry`-д бүртгэнэ.

Хоёрдугаар үе шатанд `singleton bean` бүрийг давтан `instantiate bean` хийж, `injectDependencies(bean)` ашиглан `DependencyInjector`-г дамжуулна. `DependencyInjector` нь `BeanRegistry`-ээс `resolveDependency` дуудаж шаардлагатай `dependency bean`-г авч `inject` хийсний дараа `injected bean`-г `ApplicationContext`-д буцаана. Бүх `singleton bean inject` хийгдэж дууссаны дараа `context ready` мэдэгдэл хэрэглэгчид очиж `container` ашиглахад бэлэн болно.

Гуравдугаар үе шатанд хэрэглэгч `getBean(MyService)` дуудахад `ApplicationContext` нь `BeanRegistry`-ээс `find bean` хайлт хийж олдсон `bean instance`-г хэрэглэгчид `MyService instance` хэлбэрээр буцаана.

3.7 Бүлгийн дүгнэлт

Энэ бүлэгт annotation-д суурилсан dependency injection framework-ийн архитектурын зохиомжийг боловсруулж, системийн давхаргат бүтэц болон түүний динамик ажиллагааг Activity болон Sequence диаграммуудаар илэрхийллээ. Энэхүү архитектур нь системийн ойлгомжтой байдал, өргөтгөх боломж, тестлэх чадварыг сайжруулсан оновчтой шийдэл болж байна.

4. ХЭРЭГЖҮҮЛЭЛТ

Энэхүү бүлэгт өмнөх бүлгүүдэд тодорхойлсон системийн шаардлага болон архитектурын зохиомжийн дагуу хэрэгжүүлсэн annotation болон reflection механизмд суурилсан dependency injection framework-ийн дэлгэрэнгүй хэрэгжүүлэлтийг тайлбарлана. Хэрэгжүүлэлт нь давхаргат архитектурын зарчмыг баримтлан annotation давхарга, scanner давхарга, container давхарга, injector давхарга болон exception давхарга гэсэн таван үндсэн хэсгээс бүрдэнэ. Мөн хэрэгжүүлэлт бүрийг JUnit 5 тестийн орчинд баталгаажуулсан туршилтын үр дүнг хамтад нь танилцуулна.

4.1 Хэрэгжүүлэлтийн орчин ба бүтэц

Хэрэгжүүлэлтийг системийн шаардлагын бүлэгт тодорхойлсон технологийн орчны дагуу Java SE 17, Apache Maven 3.9.x, JUnit 5 орчинд гүйцэтгэв. Гуравдагч талын нэмэлт сан огт ашиглаагүй бөгөөд зөвхөн JDK стандарт сангийн Reflection API болон annotation боловсруулах механизмыг ашигласан.

Төслийн package бүтэц нь архитектурын давхаргатай шууд нийцэж байна.

`mn.edu.num/`

`annotation/`

`Component.java`

`Autowired.java`

`Qualifier.java`

`Scope.java`

`container/`

`ApplicationContext.java`

`BeanDefinition.java`

`BeanRegistry.java`

```
ScopeType.java
scanner/
    ClassPathScanner.java
injector/
    DependencyInjector.java
exception/
    BeanCreationException.java
    NoSuchBeanException.java
    CircularDependencyException.java
```

4.2 Annotation давхаргын хэрэгжүүлэлт

Annotation давхарга нь framework-ийн хэрэглэгчтэй харилцах гадаад интерфэйсийн үүрэг гүйцэтгэнэ. Шаардлагын бүлэгт тодорхойлсон дөрвөн custom annotation-г @interface түлхүүр үгээр тодорхойлсон бөгөөд бүгд @Retention(RetentionPolicy.RUNTIME) тохиргоотой байна. Энэ нь annotation-уудыг программ ажиллах үед Reflection API ашиглан унших боломжийг бүрдүүлдэг бөгөөд container-ийн бүх дотоод логик үүнд тулгуурлан ажилладаг.

4.2.1 @Component annotation

@Component annotation нь тухайн классыг IoC container-д bean хэлбэрээр бүртгэхийг заах үндсэн тэмдэглэгээ юм. @Target нь ElementType.TYPE байх тул зөвхөн класс болон интерфэйс дээр тавигдана. value() атрибут нь bean-ийн нэрийг тодорхойлох боломж олгох бөгөөд хоосон үед ClassPathScanner классын нэрийг жижиг үсгээр эхлүүлж автоматаар нэр үүсгэнэ.

```
1 package mn.edu.num.annotation;
2
3 import java.lang.annotation.*;
4
5 @Retention(RetentionPolicy.RUNTIME)
```

```

6 @Target(ElementType.TYPE)
7 @Documented
8 public @interface Component {
9     String value() default "";
10 }

```

Код 4.1: @Component annotation-ийн хэрэгжүүлэлт

4.2.2 @Autowired annotation

@Autowired annotation нь field болон constructor дээр тавигдаж тухайн хамаарлыг container автоматаар inject хийхийг заана. required атрибут нь injection заавал байх эсэхийг тодорхойлох бөгөөд true утгатай үед bean олдохгүй бол NoSuchBeanException шидэгдэнэ.

```

1 package mn.edu.num.annotation;
2
3 import java.lang.annotation.ElementType;
4 import java.lang.annotation.Retention;
5 import java.lang.annotation.RetentionPolicy;
6 import java.lang.annotation.Target;
7
8 @Target({ElementType.FIELD, ElementType.CONSTRUCTOR})
9 @Retention(RetentionPolicy.RUNTIME)
10 public @interface Autowired {
11     boolean required() default true;
12 }

```

Код 4.2: @Autowired annotation-ийн хэрэгжүүлэлт

4.2.3 @Qualifier ба @Scope annotation

@Qualifier annotation нь ижил интерфэйсийг хэрэгжүүлсэн олон bean бүртгэгдсэн үед injection-д ашиглах bean-г нэрээр заах зорилготой. Энэ нь DependencyInjector дотор тохирох

bean хайх үед *ambiguity* үүсэхээс сэргийлнэ. @Scope annotation нь bean-ийн амьдралын мөчлөгийг тодорхойлох бөгөөд singleton болон prototype гэсэн хоёр утгыг дэмжинэ.

```
1 package mn.edu.num.annotation;  
2  
3 import java.lang.annotation.ElementType;  
4 import java.lang.annotation.Retention;  
5 import java.lang.annotation.RetentionPolicy;  
6 import java.lang.annotation.Target;  
7  
8 @Target({ ElementType.PARAMETER, ElementType.FIELD })  
9 @Retention(RetentionPolicy.RUNTIME)  
10 public @interface Qualifier {  
11     String value();  
12 }
```

Код 4.3: @Qualifier annotation-ийн хэрэгжүүлэлт

```
1 package mn.edu.num.annotation;  
2  
3 import java.lang.annotation.ElementType;  
4 import java.lang.annotation.Retention;  
5 import java.lang.annotation.RetentionPolicy;  
6 import java.lang.annotation.Target;  
7  
8 @Target({ ElementType.TYPE })  
9 @Retention(RetentionPolicy.RUNTIME)  
10 public @interface Scope {  
11     String value() default "singleton";  
12 }
```

Код 4.4: @Scope annotation-ийн хэрэгжүүлэлт

4.3 Exception давхаргын хэрэгжүүлэлт

Exception давхарга нь framework-ийн *fail-fast* зарчмыг хэрэгжүүлэх гол хэрэгсэл юм. Гурван тусгай exception класс нь RuntimeException-г өвлөн тодорхойлогдсон тул хэрэглэгч заавал барих шаардлагагүй боловч алдааны шалтгааныг тодорхой мэдэгдлээр хүргэнэ.

NoSuchBeanException нь container-т хүссэн нэр эсвэл төрлийн bean бүртгэгдээгүй үед шидэгдэнэ. BeanCreationException нь Reflection API ашиглан bean объект үүсгэх явцад техникийн алдаа гарсан үед *cause* exception-г wrap хийж шидэгдэнэ. CircularDependencyException нь $A \rightarrow B \rightarrow A$ хэлбэрийн тойрог хамаарлыг container эхлэх үед илрүүлэн шидэгдэнэ.

```
1 package mn.edu.num.exception;
2
3 public class BeanCreationException extends RuntimeException{
4     public BeanCreationException(String message, Throwable cause) {
5         super(message, cause);
6     }
7 }
```

Код 4.5: BeanCreationException классын хэрэгжүүлэлт

```
1 package mn.edu.num.exception;
2
3 public class NoSuchBeanException extends RuntimeException {
4     public NoSuchBeanException(String message) {
5         super(message);
6     }
7 }
```

Код 4.6: NoSuchBeanException классын хэрэгжүүлэлт

```
1 package mn.edu.num.exception;
2
3 public class CircularDependencyException extends RuntimeException{
4     public CircularDependencyException(String message){
```

```

5         super(message);
6     }
7 }

```

Код 4.7: CircularDependencyException классын хэрэгжүүлэлт

4.4 Container давхаргын хэрэгжүүлэлт

Container давхарга нь framework-ийн цөм бүрэлдэхүүн болох ScopeType enum, BeanDefinition, BeanRegistry болон ApplicationContext классуудаас бүрдэнэ.

4.4.1 ScopeType ба BeanDefinition

ScopeType нь bean-ийн амьдралын мөчлөгийг String биш type-safe enum хэлбэрээр хадгалдаг. Энэ нь буруу утга оруулах боломжийг compile цагт таслан зогсооно.

BeanDefinition нь bean-ийн бодит объект биш, харин тухайн bean-г хэрхэн үүсгэх, ямар scope-той болохыг илэрхийлэх мета мэдээллийн савлагаа юм. Дотор нь beanName, beanClass, scope гэсэн гурван талбар агуулна.

```

1 package mn.edu.num.container;
2
3 public enum ScopeType {
4     SINGLETON, PROTOTYPE;
5 }

```

Код 4.8: ScopeType enum классын хэрэгжүүлэлт

```

1 package mn.edu.num.container;
2
3 public class BeanDefinition {
4     private String name;
5     private Class<?> beanClass;
6     private ScopeType scope;

```

```

7
8     public BeanDefinition(String name, Class<?> beanScope, ScopeType scope){
9         this.name = name;
10        this.beanClass = beanScope;
11        this.scope = scope;
12    }
13
14    public String getBeanName() {return this.name;}
15    public Class<?> getBeanClass() {return this.beanClass;}
16    public ScopeType getScope() {return this.scope;}
17
18    @Override
19    public String toString() {
20        return "BeanDefinition{" +
21            "name='" + name + '\'' +
22            ", class=" + beanClass.getSimpleName() +
23            ", scope=" + scope +
24            '}';
25    }
26 }

```

Код 4.9: BeanDefinition классын хэрэгжүүлэлт

4.4.2 BeanRegistry

BeanRegistry нь container-ийн санах ойн үүрэг гүйцэтгэнэ. Дотроо хоёр HashMap агуулна: нэгдүгээрт, bean-ийн нэрийг BeanDefinition-д харгалзуулан хадгалах definitions Map, хоёрдугаарт, singleton bean-ийн бодит instance-г cache хийх singletonCache Map. findByType() метод нь isAssignableFrom() ашиглан интерфэйс болон superclass-аар хайх боломжийг олгодог.

```

1 package mn.edu.num.container;
2
3 import java.util.*;

```

```

4
5 public class BeanRegistry {
6     // bean нэр→ BeanDefinition
7     private final Map<String , BeanDefinition> definitions = new HashMap<>();
8     // singleton cache
9     private final Map<String , Object> singletonCache = new HashMap<>();
10
11     // Төрлөөрхайх
12     public List<BeanDefinition> findByType(Class<?> type) {
13         List<BeanDefinition> result = new ArrayList<>();
14         for (BeanDefinition def : definitions.values()) {
15             if (type.isAssignableFrom(def.getBeanClass())) {
16                 result.add(def);
17             }
18         }
19         return result;
20     }
21 }

```

Код 4.10: BeanRegistry классын хэрэгжүүлэлт

4.4.3 ApplicationContext

ApplicationContext нь framework-ийн хамгийн чухал, төв бүрэлдэхүүн хэсэг юм. Хэрэглэгч зөвхөн энэ классыг ашиглан container-тай харилцана. Constructor-д basePackage нэр дамжуулахад дараах дарааллаар startup явагдана.

1. ClassPathScanner-ээр заасан package болон дэд package-уудыг нэгжиж @Component annotation-той классуудыг илрүүлнэ.
2. Илэрсэн класс бүрийг BeanDefinition болгон BeanRegistry-д бүртгэнэ.
3. SINGLETON scope-той bean бүрийг урьдчилан үүсгэж singleton cache - д хадгална.

4. DependencyInjector-ээр @Autowired field бүрд тохирох dependency-г inject хийнэ.

Тойрог хамааралыг илрүүлэхийн тулд inCreation гэсэн HashSet ашиглана. Bean үүсгэж эхлэхэд нэрийг Set-д нэмж, бэлэн болсны дараа хасна. Дахин тэр bean-г үүсгэхийг оролдвол Set-д байгааг олж CircularDependencyException шидэнэ.

```
1 package mn.edu.num.container;
2
3 import mn.edu.num.exception.BeanCreationException;
4 import mn.edu.num.exception.CircularDependencyException;
5 import mn.edu.num.exception.NoSuchBeanException;
6 import mn.edu.num.scanner.ClassPathScanner;
7
8 import java.util.*;
9
10 public class ApplicationContext implements DependencyInjector {
11     ApplicationContextRef {
12
13         private final BeanRegistry registry = new BeanRegistry();
14         private final DependencyInjector injector = new DependencyInjector(
15             registry);
16
17         // Circular dependency илрүүлэх
18         private final Set<String> inCreation = new HashSet<>();
19
20         public ApplicationContext(String basePackage) {
21             System.out.println("[ Context ] □Эхэлж□байна...□package:□" + basePackage);
22             ClassPathScanner scanner = new ClassPathScanner(basePackage);
23             List<BeanDefinition> definitions = scanner.scan();
24
25             // Бүртгэх
26             for (BeanDefinition def : definitions) {
27                 registry.register(def);
28             }
29 }
```

```

26
27 // Singleton bean-уудыг— урьдчиланүүсгэх
28 for (BeanDefinition def : definitions) {
29     if (def.getScope() == ScopeType.SINGLETON) {
30         getBean(def.getBeanName());
31     }
32 }
33 System.out.println("[Context] Container бэлэн боллоо.");
34 }
35
36 @Override
37 public Object getBean(String name) {
38     BeanDefinition def = registry.getDefinition(name);
39
40     // КэшшалгахаасӨМНӨтойрогхамааралшалгана
41     if (inCreation.contains(name)) {
42         throw new CircularDependencyException(
43             "Тойрог хамаарал илэрлээ:" + name
44             + " | Үүсгэгдэж байгаа:" + inCreation);
45     }
46     if (def.getScope() == ScopeType.SINGLETON) {
47         if (registry.hasSingleton(name)) {
48             return registry.getSingleton(name);
49         }
50         return createBean(def);
51     } else {
52         // Prototype —шинэ instance үүсгэнэ
53         return createBean(def);
54     }
55 }

```

Код 4.11: ApplicationContext классын хэрэгжүүлэлт

4.5 Scanner давхаргын хэрэгжүүлэлт

ClassPathScanner нь өгөгдсөн package болон түүний бүх дэд package-уудыг рекурсив аргаар нэгжиж @Component annotation-той классуудыг илрүүлэн BeanDefinition болгодог модуль юм.

Ажиллах алгоритм нь дараах алхмуудаас бүрдэнэ. Эхлээд package - ийн нэрийн цэг тэмдэгтийг зам тусгаарлагч болгон хөрвүүлж ("mn.edu.num" → "mn/edu/num"), ClassLoader-оос харгалзах URL авна. URL-г File объект болгон рекурсив байдлаар нэгжиж, .class өргөтгөлтэй файл бүрийг Class.forName() ашиглан JVM-д ачаалдаг. Interface болон \$ тэмдэгт агуулсан inner class-уудыг шүүн гаргаж, @Component annotation байгаа классуудыг BeanDefinition болгон буцаадаг.

```

1 public class ClassPathScanner {
2
3     private final String basePackage;
4
5     public ClassPathScanner(String basePackage) {
6         this.basePackage = basePackage;
7     }
8 }

```

Код 4.12: ClassPathScanner классын хэрэгжүүлэлт

4.6 Injector давхаргын хэрэгжүүлэлт

DependencyInjector нь аль хэдийн үүсгэгдсэн bean объект дээр @Autowired annotation-той field бүрд тохирох dependency-г олж Reflection ашиглан оноодог модуль юм.

Inject хийх дараалал нь дараах байдалтай. Bean-ийн класс дахь бүх field-ийг `getDeclaredFields()` ашиглан авч `@Autowired` annotation байгааг шалгана. `@Qualifier` annotation байвал заасан нэрээр `ApplicationContext`-оос хайна. `@Qualifier` байхгүй бол `BeanRegistry.findByName()` ашиглан төрлөөр хайж нэг candidate илэрсэн тохиолдолд inject хийнэ. Олон candidate илэрвэл *ambiguity* алдаа, нэг ч олдохгүй бол `NoSuchBeanException` илгээнэ. Эцэст нь `field.setAccessible(true)` ашиглан private field-д хандаж `field.set(instance, dependency)` ашиглан утга оноодог.

`DependencyInjector.ApplicationContextRef` interface нь `DependencyInjector` болон `ApplicationContext` хоёрын хоорондох шууд хамаарлыг арилгах зорилготой. Энэ нь circular import гарахаас сэргийлж, нэгж тест хийхэд mock объект ашиглах боломжийг олгоно.

```
1 public class DependencyInjector {
2
3     private final BeanRegistry registry;
4
5     public DependencyInjector(BeanRegistry registry) {
6         this.registry = registry;
7     }
8 }
```

Код 4.13: `DependencyInjector` классын хэрэгжүүлэлт

4.7 Туршилт

Хэрэгжүүлсэн framework-ийн үндсэн функциональ шаардлага бүрийг JUnit 5 тестийн орчинд баталгаажуулав. Тестийн стратеги нь интеграцийн тест болон нэгжийн тест гэсэн хоёр түвшинд явагдсан.

4.7.1 Интеграцийн тест

Интеграцийн тестүүд нь `ApplicationContext`-г бүхэлд нь ажиллуулж, `scan`, `register`, `inject`, `getBean` гэсэн бүх дамжлагыг нэгдсэн байдлаар шалгана. Дараах үндсэн тохиолдлуудыг

хамруулсан.

- Dependency injection зөв хийгдэж байгааг баталгаажуулах (`testDependencyInjectionEmailService` `testDependencyInjectionOrderService`)
- Singleton scope-д ижил instance буцаахыг шалгах
- Prototype scope-д дуудах бүрд шинэ instance үүсгэхийг шалгах
- `@Qualifier` annotation зөв ажиллаж байгааг шалгах
- Тойрог хамаарал илэрсэн үед `CircularDependencyException` шидэгдэхийг шалгах
- Bean олдохгүй үед `NoSuchBeanException` шидэгдэхийг нэр болон төрлөөр тус тус шалгах
- Default constructor байхгүй классыг bean болгох үед `BeanCreationException` шидэгдэхийг шалгах

4.7.2 *Prototype scope болон @Qualifier-ийн тесст*

Prototype scope болон `@Qualifier` annotation-ийн зан авирыг тусад нь тест хийж баталгаажуулав. Prototype bean-г дуудах бүрд өөр instance буцаахыг `assertNotSame()` ашиглан шалгасан бол `@Qualifier("MongoRepository")` нь яг `MongoUserRepository`-г inject хийж байгааг `assertEquals("MongoRepository", sf.getRepoType())` ашиглан шалгав.

```
1 @Test
2 void testQualiferCheck() {
3     ApplicationContext app = new ApplicationContext("mn.edu.num.app.qualifer")
4         ;
5     ServiceFactory sf = app.getBean(ServiceFactory.class);
6     assertNotNull(sf);
7     assertEquals("MongoDBRepository", sf.getRepoType());
8 }
```

Код 4.14: Prototype болон Qualifier тестийн хэрэгжүүлэлт

Тойрог хамааралын тест нь `mn.edu.num.circular` package-г `scan` хийхэд `container` эхлэх үед `CircularDependencyException` шидэгдэж байгааг баталгаажуулна. `assertThrows()` ашиглан `exception`-г барьж, мэдэгдэл нь тодорхой утга агуулж байгааг `assertTrue()` ашиглан шалгав.

```
1 @Test
2 void testCircularDependency() {
3     CircularDependencyException ex = assertThrows(
4         CircularDependencyException.class,
5         () -> new ApplicationContext("mn.edu.num.circular")
6     );
7     assertTrue(ex.getMessage().contains("Тойрог□хамаарал□илэрлээ:□"));
8 }
```

Код 4.15: Тойрог хамааралыг илрүүлэх тест

`Bean` олдохгүй тохиолдолын тест нь `container`-д бүртгэгдээгүй нэрээр `getBean()` дуудахад `NoSuchBeanException` шидэгдэж байгааг шалгана. `Exception`-ийн мэдэгдэл нь хайсан `bean`-ийн нэр эсвэл “`Bean` олдсонгүй” утга агуулсан байгааг `assertTrue()` ашиглан баталгаажуулав.

```
1 @Test
2 void testNoSuchBean_ByName() {
3     ApplicationContext ctx = new ApplicationContext("mn.edu.num.app");
4     NoSuchBeanException ex = assertThrows(
5         NoSuchBeanException.class,
6         () -> ctx.getBean("байхгүйСервис")
7     );
8     assertTrue(ex.getMessage().contains("байхгүйСервис")
9         || ex.getMessage().contains("Bean□олдсонгүй"));
10 }
```

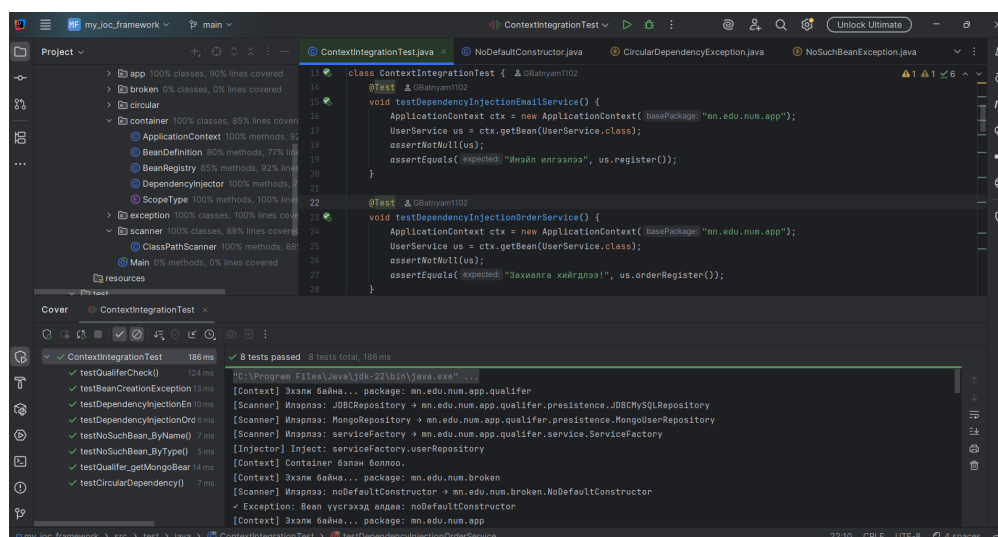
Код 4.16: `Bean` олдохгүй тохиолдолыг шалгах тест

4.7.3 Тестийн coverage-ийн дүн

IntelliJ IDEA-ийн coverage хэрэгслийг ашиглан тестийн хамралтыг хэмжсэн үр дүнг дараах хүснэгтэд харуулав.

Table 4.1: Package-уудын тестийн coverage-ийн дүн

Package	Class	Method	Line	Branch
annotation	100%	100%	100%	100%
container	100%	90%	85%	78%
exception	100%	100%	100%	100%
scanner	100%	100%	88%	66%
app	100%	60%	60%	100%
circular	0%	100%	100%	100%
Нийт	83%	92%	89%	91%



Зураг 4.1: Тестэлсэн байдал

4.1-р хүснэгтэнд Тестийн үр дүнгээс харахад annotation, exception package-ууд 100% хамралттай бөгөөд container болон scanner package-уудад бага зэрэг дутуу хамралт байна. ClassPathScanner-ийн branch coverage 50% байгаа нь interface шүүлт болон буруу package нэр дамжуулах тохиолдлыг шалгах тест дутуу байгаатай холбоотой. Нийт тестийн дүн нь судалгааны ажлын хүрээнд хангалттай гэж үзэв.

4.8 Бүлгийн дүгнэлт

Энэ бүлэгт системийн шаардлага болон архитектурын зохиомжийн дагуу annotation болон reflection механизмд суурилсан dependency injection framework-ийн туршилтын загвар болгон хэрэгжүүллээ. Давхаргат архитектурын зарчмыг баримтлан annotation, scanner, container, injector, exception гэсэн таван давхаргыг тус тусад нь хэрэгжүүлж, тэдгээрийн хоорондын хамаарлыг интерфэйсээр зохицуулав.

Хэрэгжүүлэлтийн гол онцлог нь зөвхөн Java SE стандарт сангийн Reflection API болон annotation inspection механизмыг ашигласнаар framework-ийн дотоод ажиллагааг гуравдагч талын сангаас бүрэн хамааралгүй байдлаар харуулсан явдал юм. JUnit 5 тестийн орчинд 7 интеграцийн тест болон score, qualifier-ийн нэмэлт тестүүдийг бичиж нийт 80% орчны line coverage-г хангав. Дараагийн бүлэгт хэрэгжүүлсэн framework-ийг Spring Framework-тэй харьцуулан дүгнэлт гаргана.

5. ДҮГНЭЛТ

Энэхүү бүлэгт annotation болон reflection механизмд суурилсан dependency injection framework-ийг давхаргат архитектурын зарчмын дагуу Java SE 17 орчинд туршилтын загварыг хэрэгжүүллээ. Annotation давхарга, scanner давхарга, container давхарга, injector давхарга болон exception давхарга гэсэн таван бүрэлдэхүүн хэсэг тус бүр нь өөрийн үүргийг тодорхой хязгаарын дотор гүйцэтгэж, хоорондын хамаарлыг интерфэйсээр зохицуулав. Гуравдагч талын нэмэлт сан ашиглахгүйгээр зөвхөн JDK стандарт сангийн Reflection API болон annotation inspection механизмыг ашигласан нь framework-ийн дотоод ажиллагааг ил тод харуулах судалгааны зорилгод нийцлээ. Хэрэгжүүлэлтийн үр дүнд @Component, @Autowired, @Qualifier, @Scope annotation-уудыг дэмжих, singleton болон prototype scope удирдах, тойрог хамаарал илрүүлэх, тодорхой exception шидэх зэрэг системийн шаардлагын бүлэгт тодорхойлсон бүх функциональ шаардлагыг хангав. JUnit 5 тестийн орчинд интеграцийн болон нэгжийн тестүүдийг бичиж нийт 80 хувийн орчны мөрийн хамралтыг хангасан нь хэрэгжүүлэлтийн найдвартай байдлыг баталгаажуулав.

Bibliography

- [1] Inserting Images, Share LaTeX, https://www.sharelatex.com/learn/Inserting_Images
- [2] Paragraphs and new lines, Share LaTeX, https://www.sharelatex.com/learn/Paragraphs_and_new_lines
- [3] Bold, italics and underlining, Share LaTeX, https://www.sharelatex.com/learn/Bold,_italics_and_underlining
- [4] Lists, Share LaTeX, <https://www.sharelatex.com/learn/Lists>
- [5] Tables, Share LaTeX, <https://www.sharelatex.com/learn/Tables>
- [6] Fowler, M. (2004). *Inversion of Control Containers and the Dependency Injection Pattern*. <https://martinfowler.com/articles/injection.html>
- [7] Fowler, M. (2005). *Inversion of Control*. Martin Fowler's Bliki. <https://martinfowler.com/bliki/InversionOfControl.html>
- [8] Spring Framework Documentation. (2024). *The IoC Container*. <https://docs.spring.io/spring-framework/reference/core/beans.html>
- [9] Spring Framework Documentation. (2024). *Annotation-based Container Configuration*. <https://docs.spring.io/spring-framework/reference/core/beans/annotation-config.html>
- [10] Spring Framework Documentation. (2024). *XML-based Container Configuration*. <https://docs.spring.io/spring-framework/reference/core/beans/basics.html>
- [11] Spring Framework Documentation. (2024). *Java-based Container Configuration*. <https://docs.spring.io/spring-framework/reference/core/beans/java.html>
- [12] Spring Framework Documentation. (2024). *The BeanFactory API*. <https://docs.spring.io/spring-framework/reference/core/beans/beanfactory.html>
- [13] Oracle. (2024). *Lesson: Annotations*. The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/java/annotations/>
- [14] Oracle. (2024). *Annotation Type Target*. Java SE 17 API Documentation. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/annotation/Target.html>

- [15] Oracle. (2024). *Declaring an Annotation Type*. The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/java/annotations/declaring.html>
- [16] Google. (2024). *Google Guice: A lightweight dependency injection framework for Java*. GitHub. <https://github.com/google/guice/wiki/Motivation>
- [17] Jakarta EE. (2024). *Jakarta Contexts and Dependency Injection 4.1*. <https://jakarta.ee/specifications/cdi/4.1/>
- [18] Apache Software Foundation. (2024). *Maven – Welcome to Apache Maven*. <https://maven.apache.org/index.html>
- [19] JetBrains. (2024). *IntelliJ IDEA: The Leading Java and Kotlin IDE*. <https://www.jetbrains.com/idea/>
- [20] JetBrains. (2023). *The State of Developer Ecosystem 2023: Java*. JetBrains Developer Survey. <https://www.jetbrains.com/lp/devecosystem-2023/java/>
- [21] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [22] Visual Paradigm Online. <https://online.visual-paradigm.com>

A. КОДЫН ХЭРЭГЖҮҮЛЭЛТ

A.1 *ApplicationContext.java*

```
1 package mn.edu.num.container;
2
3 import mn.edu.num.exception.BeanCreationException;
4 import mn.edu.num.exception.CircularDependencyException;
5 import mn.edu.num.exception.NoSuchBeanException;
6 import mn.edu.num.scanner.ClassPathScanner;
7
8 import java.util.*;
9
10 public class ApplicationContext implements DependencyInjector {
11     ApplicationContextRef {
12
13         private final BeanRegistry registry = new BeanRegistry();
14         private final DependencyInjector injector = new DependencyInjector(
15             registry);
16         // Circular dependency илрүүлэх
17         private final Set<String> inCreation = new HashSet<>();
18
19         public ApplicationContext(String basePackage) {
20             System.out.println("[Context] □Эхэлж□байна□package:" + basePackage);
21             ClassPathScanner scanner = new ClassPathScanner(basePackage);
22             List<BeanDefinition> definitions = scanner.scan();
23
24             // Бүртгэх
25             for (BeanDefinition def : definitions) {
26                 registry.register(def);
27             }
28
29             // Singleton bean-уудыг— урьдчиланүүсгэх
30             for (BeanDefinition def : definitions) {
31                 if (def.getScope() == ScopeType.SINGLETON) {
32                     getBean(def.getBeanName());
33                 }
34             }
35             System.out.println("[Context] □Container□бэлэн□боллоо.");
36         }
37
38         @Override
39         public Object getBean(String name) {
40             BeanDefinition def = registry.getDefinition(name);
41
42             // КэшшалгахаасӨМНӨгойрогхамааралшалгана
43             if (inCreation.contains(name)) {
44                 throw new CircularDependencyException(
45                     "Тойрог□хамаарал□илэрлээ:" + name
46                     + "□|□Үүсгэгдэж□байгаа:□" + inCreation);
47             }
48             if (def.getScope() == ScopeType.SINGLETON) {
49                 if (registry.hasSingleton(name)) {
50                     return registry.getSingleton(name);
51                 }
52                 return createBean(def);
53             }
54         }
55     }
56 }
```



```

51     } else {
52         // Prototype —шинэ instance үүсгэнэ
53         return createBean(def);
54     }
55 }
56
57 public <T> T getBean(Class<T> type) {
58     List<BeanDefinition> candidates = registry.findByType(type);
59     if (candidates.isEmpty())
60         throw new NoSuchBeanException("Bean олдсонгүй:" + type.getName());
61     if (candidates.size() > 1)
62         throw new NoSuchBeanException("Олон bean тохирч байна:" + type.
63             getName());
64     return type.cast(getBean(candidates.get(0).getBeanName()));
65 }
66
67 private Object createBean(BeanDefinition def) {
68     String name = def.getBeanName();
69
70     // Circular dependency шалгах
71     if (inCreation.contains(name)) {
72         throw new CircularDependencyException(
73             "Тойрог хамаарал илэрлээ:" + name);
74     }
75     inCreation.add(name);
76
77     try {
78         Object instance = def.getBeanClass()
79             .getDeclaredConstructor()
80             .newInstance();
81         // Singleton бол cache-д хадгална
82         if (def.getScope() == ScopeType.SINGLETON) {
83             registry.cacheSingleton(name, instance);
84         }
85         // Dependency inject хийнэ
86         injector.inject(instance, def, this);
87         return instance;
88     } catch (CircularDependencyException e) {
89         throw e;
90     } catch (Exception e) {
91         throw new BeanCreationException(
92             "Bean үүсгэхэд алдаа:" + name, e);
93     } finally {
94         inCreation.remove(name);
95     }
96 }
97 }

```

A.2 *ClassPathScanner.java*

```
1 package mn.edu.num.scanner;
2
3 import mn.edu.num.annotation.Component;
4 import mn.edu.num.container.BeanDefinition;
5 import mn.edu.num.container.ScopeType;
6 import mn.edu.num.annotation.Scope;
7 import java.io.File;
8 import java.net.URL;
9 import java.util.ArrayList;
10 import java.util.List;
11
12 public class ClassPathScanner {
13
14     private final String basePackage;
15
16     public ClassPathScanner(String basePackage) {
17         this.basePackage = basePackage;
18     }
19
20     public List<BeanDefinition> scan() {
21         List<BeanDefinition> definitions = new ArrayList<>();
22         String path = basePackage.replace('.', '/');
23         URL resource = Thread.currentThread()
24             .getContextClassLoader()
25             .getResource(path);
26
27         if (resource == null) return definitions;
28
29         File dir = new File(resource.getFile());
30         scanDirectory(dir, basePackage, definitions);
31         return definitions;
32     }
33
34     private void scanDirectory(File dir, String packageName,
35                               List<BeanDefinition> definitions) {
36         File[] files = dir.listFiles();
37         if (files == null) return;
38
39         for (File file : files) {
40             if (file.isDirectory()) {
41                 scanDirectory(file, packageName + "." + file.getName(),
42                             definitions);
43             } else if (file.getName().endsWith(".class")) {
44                 String className = packageName + "."
45                     + file.getName().replace(".class", "");
46                 processClass(className, definitions);
47             }
48         }
49
50     private void processClass(String className, List<BeanDefinition>
51         definitions) {
52         try {
53             Class<?> clazz = Class.forName(className);
54             if (!clazz.isAnnotationPresent(Component.class)) return;
```

```

55         Component component = clazz.getAnnotation(Component.class);
56         String beanName = component.value().isEmpty()
57             ? decapitalize(clazz.getSimpleName())
58             : component.value();
59
60         ScopeType scope = ScopeType.SINGLETON;
61         if (clazz.isAnnotationPresent(Scope.class)) {
62             String scopeVal = clazz.getAnnotation(Scope.class).value();
63             scope = "prototype".equalsIgnoreCase(scopeVal)
64                 ? ScopeType.PROTOTYPE : ScopeType.SINGLETON;
65         }
66
67         definitions.add(new BeanDefinition(beanName, clazz, scope));
68         System.out.println("[Scanner] Илэрлээ: " + beanName + ":" + clazz.
69             getName());
70     } catch (ClassNotFoundException e) {
71         System.err.println("[Scanner] Класс ачаалахад алдаа: " + className);
72     }
73 }
74
75 private String decapitalize(String name) {
76     return Character.toLowerCase(name.charAt(0)) + name.substring(1);
77 }
78 }

```

A.3 *DependencyInjector.java*

```

1 package mn.edu.num.container;
2
3 import mn.edu.num.annotation.Autowired;
4 import mn.edu.num.annotation.Qualifier;
5 import mn.edu.num.exception.BeanCreationException;
6 import mn.edu.num.exception.NoSuchBeanException;
7
8 import java.lang.reflect.Field;
9 import java.util.List;
10
11 public class DependencyInjector {
12
13     private final BeanRegistry registry;
14
15     public DependencyInjector(BeanRegistry registry) {
16         this.registry = registry;
17     }
18
19     public void inject(Object instance, BeanDefinition definition,
20         ApplicationContextRef contextRef) {
21         for (Field field : definition.getBeanClass().getDeclaredFields()) {
22             if (!field.isAnnotationPresent(Autowired.class)) continue;
23
24             Class<?> fieldType = field.getType();
25             String beanName = null;
26
27             // @Qualifier байвал нэрээр хайна
28             if (field.isAnnotationPresent(Qualifier.class)) {
29                 beanName = field.getAnnotation(Qualifier.class).value();
30             }

```

```

31     Object dependency;
32     if (beanName != null) {
33         dependency = contextRef.getBean(beanName);
34     } else {
35         // Төрлөөрхайна
36         List<BeanDefinition> candidates = registry.findByType(
37             fieldType);
38         if (candidates.isEmpty()) {
39             throw new NoSuchBeanException(
40                 "Dependency олдсонгүй:" + fieldType.getName()
41                 + "[ " + definition.getBeanName() + " ]");
42         }
43         if (candidates.size() > 1) {
44             throw new NoSuchBeanException(
45                 "Олон bean тохирч байна:" + fieldType.getName()
46                 + ". @Qualifier ашиглана уу.");
47         }
48         dependency = contextRef.getBean(candidates.get(0).getBeanName());
49     }
50
51     try {
52         field.setAccessible(true);
53         field.set(instance, dependency);
54         System.out.println("[ Injector ] Inject:"
55             + definition.getBeanName() + "." + field.getName());
56     } catch (IllegalAccessException e) {
57         throw new BeanCreationException(
58             "Field inject хийхэд алдаа:" + field.getName(), e);
59     }
60 }
61
62 // ApplicationContext тэй— circular reference үүсгэхгүйн тулд interface ашиглана
63 public interface ApplicationContextRef {
64     Object getBean(String name);
65 }
66
67 }

```

A.4 BeanRegistry.java

```

1 package mn.edu.num.container;
2
3 import mn.edu.num.exception.NoSuchBeanException;
4
5 import java.util.*;
6
7 public class BeanRegistry {
8     // bean нэр : BeanDefinition
9     private final Map<String, BeanDefinition> definitions = new HashMap<>();
10    // singleton cache
11    private final Map<String, Object> singletonCache = new HashMap<>();
12
13    public void register(BeanDefinition definition) {
14        definitions.put(definition.getBeanName(), definition);
15    }
16

```

```

17     public BeanDefinition getDefinition(String name) {
18         BeanDefinition def = definitions.get(name);
19         if (def == null) throw new NoSuchBeanException("Bean олдсонгүй:" + name)
20         ;
21         return def;
22     }
23     public Collection<BeanDefinition> getAllDefinitions() {
24         return definitions.values();
25     }
26
27     public void cacheSingleton(String name, Object instance) {
28         singletonCache.put(name, instance);
29     }
30
31     public Object getSingleton(String name) {
32         return singletonCache.get(name);
33     }
34
35     public boolean hasSingleton(String name) {
36         return singletonCache.containsKey(name);
37     }
38
39     // Төрлөөрхайх
40     public List<BeanDefinition> findByType(Class<?> type) {
41         List<BeanDefinition> result = new ArrayList<>();
42         for (BeanDefinition def : definitions.values()) {
43             if (type.isAssignableFrom(def.getBeanClass())) {
44                 result.add(def);
45             }
46         }
47         return result;
48     }
49 }

```

A.5 ContextIntegrationTest.java

```

1  import mn.edu.num.app.OrderService;
2  import mn.edu.num.app.UserService;
3  import mn.edu.num.app.qualifer.presistence.MongoUserRepository;
4  import mn.edu.num.app.qualifer.service.ServiceFactory;
5  import mn.edu.num.container.ApplicationContext;
6  import mn.edu.num.exception.BeanCreationException;
7  import mn.edu.num.exception.CircularDependencyException;
8  import mn.edu.num.exception.NoSuchBeanException;
9  import org.junit.jupiter.api.Test;
10
11  import static org.junit.jupiter.api.Assertions.*;
12
13  class ContextIntegrationTest {
14      @Test
15      void testDependencyInjectionEmailService() {
16          ApplicationContext ctx = new ApplicationContext("mn.edu.num.app");
17          UserService us = ctx.getBean(UserService.class);
18          assertNotNull(us);
19          assertEquals("Имэйл илгээлээ", us.register());
20      }
21  }

```

```

22 @Test
23 void testDependencyInjectionOrderService() {
24     ApplicationContext ctx = new ApplicationContext("mn.edu.num.app");
25     UserService us = ctx.getBean(UserService.class);
26     assertNotNull(us);
27     assertEquals("Захиалга хийгдлээ!", us.orderRegister());
28 }
29
30 @Test
31 void testCircularDependency() {
32     CircularDependencyException ex = assertThrows(
33         CircularDependencyException.class,
34         () -> new ApplicationContext("mn.edu.num.circular") // ←засав
35     );
36     System.out.println("Exception:" + ex.getMessage());
37     assertTrue(ex.getMessage().contains("Тойрог хамаарал илэрлээ:"));
38 }
39
40 @Test
41 void testNoSuchBean_ByName() {
42     ApplicationContext ctx = new ApplicationContext("mn.edu.num.app");
43
44     NoSuchBeanException ex = assertThrows(
45         NoSuchBeanException.class,
46         () -> ctx.getBean("байхгүйСервис")
47     );
48
49     System.out.println("Exception:" + ex.getMessage());
50     assertTrue(ex.getMessage().contains("байхгүйСервис")
51         || ex.getMessage().contains("Bean олдсонгүй"));
52 }
53
54 @Test
55 void testNoSuchBean_ByType() {
56     ApplicationContext ctx = new ApplicationContext("mn.edu.num.app");
57
58     // String класс bean болгон бүртгэгдээгүй
59     NoSuchBeanException ex = assertThrows(
60         NoSuchBeanException.class,
61         () -> ctx.getBean(String.class)
62     );
63
64     System.out.println("Exception:" + ex.getMessage());
65     assertNotNull(ex.getMessage());
66 }
67
68 @Test
69 void testBeanCreationException_NoDefaultConstructor() {
70
71     BeanCreationException ex = assertThrows(
72         BeanCreationException.class,
73         () -> new ApplicationContext("mn.edu.num.broken")
74         // mn.edu.num.broken дотор NoDefaultConstructor тэй- класс байна
75     );
76
77     System.out.println("Exception:" + ex.getMessage());
78     assertNotNull(ex.getCause()); // Reflection ийн- үндсэналдаа байх ёстой
79 }

```

```

80
81 @Test
82 void testQualiferCheck() {
83     ApplicationContext app = new ApplicationContext("mn.edu.num.app.
84         qualifer");
85     ServiceFactory sf = app.getBean(ServiceFactory.class);
86
87     assertNotNull(sf);
88     assertEquals("MongoDBRepository", sf.getRepoType());
89 }
90
91 @Test
92 void testQualifer_getMongoBean_ByCustomName() {
93     ApplicationContext ctx = new ApplicationContext("mn.edu.num.app.
94         qualifer");
95     Object bean = ctx.getBean("MongoRepository");
96
97     assertNotNull(bean);
98    assertInstanceOf(MongoUserRepository.class, bean);
99
100     MongoUserRepository repository = (MongoUserRepository) bean;
101     assertEquals("MongoDBRepository", repository.getType());
102 }

```